Leios technical design and implementation plan

Sebastian Nagel sebastian.nagel@iohk.io
Nicolas Frisby nick.frisby@iohk.io
Thomas Vellekoop thomas.vellekoop@iohk.io
Michael Karg michael.karg@iohk.io
Martin Kourim martin.kourim@iohk.io
Marcin Wójtowicz marcin.wojtowicz@iohk.io
Drazen Popovic drazen.popovic@iohk.io

Contents

1 Introduction **2**

2 Overview **3**
2.1 From research to implementation . . . 4
2.2 Cardano node as a real-time system . . . 4
2.3 Concurrency and resource management . . . 5
2.4 Designing for the worst-case . . . 6
2.5 Implementation imperatives . . . 6

3 Implementation plan **7**
3.1 Approach . . . 7
3.2 Correctness in two dimensions . . . 8
3.3 Simulation and protocol validation . . . 9
3.4 Prototyping and adversarial testing . . . 10
3.5 Public testnets and integration . . . 11
3.6 Mainnet deployment readiness . . . 12

4 Dependencies and interactions **12**
4.1 On-disk storage of ledger state . . . 13
4.2 Synergies with Peras . . . 13
4.3 Era and hard-fork coordination . . . 14
4.4 Interactions with Genesis . . . 15
4.5 Impact on Mithril . . . 16

5 Risks and mitigations **16**
5.1 Key threats . . . 17
5.1.1 Data withholding . . . 17
5.1.2 Protocol bursts . . . 18
5.2 Assumptions to validate early . . . 20

6 Technical design **20**
6.1 Architecture . . . 22
6.1.1 Design guidelines . . . 23
6.2 Resource management . . . 25
6.3 Network . . . 26
6.3.1 Message latencies . . . 27
6.3.2 Traffic prioritization . . . 27
6.3.3 High-throughput transaction submission . . . 28
6.3.4 New mini-protocols . . . 29
6.4 Consensus . . . 29
6.4.1 High-throughput mempool . . . 29
6.4.2 Block production . . . 35
6.4.3 Endorser block diffusion . . . 36
6.4.4 Endorser block storage . . . 45
6.4.5 Transaction cache . . . 47
6.4.6 Voting and certification . . . 48
6.4.7 Chain selection . . . 49
6.4.8 Block validation . . . 50
6.4.9 Catching up . . . 52
6.5 Ledger . . . 52
6.5.1 Transaction validation levels . . . 53
6.5.2 New protocol parameters . . . 53
6.5.3 Key registration and rotation . . . 54
6.5.4 Committee selection . . . 56
6.5.5 Certificate verification . . . 58
6.5.6 Serialization . . . 59
6.5.7 Cryptography . . . 60
6.6 Node API . . . 62
6.6.1 LocalStateQuery additions . . . 62
6.6.2 Node-to-client . . . 62
6.6.3 Feature flags and configuration . . . 62
6.7 End-to-end testing . . . 63
6.7.1 New end-to-end tests for Leios . . . 63
6.7.2 New automated upgrade testing test suite . . . 63
6.8 Performance and tracing . . . 64
6.8.1 Observability as a first-class citizen . . . 64
6.8.2 Testing during development . . . 64
6.8.3 Testing a full implementation . . . 65

7 Glossary **66**

8 References **66**

1 Introduction

This technical design document bridges the gap between the protocol-level specification (CIP-164) and its concrete implementation in `cardano-node`. While CIP-164 defines *what* the Leios

protocol is and *why* it benefits Cardano, this document addresses *how* to implement it reliably and serve as a practical guide for implementation teams.

This document builds on the conducted impact analysis and threat modeling. The document outlines the necessary architecture changes, highlights key risks and mitigation strategies, and proposes an implementation roadmap. As the implementation plan itself contains exploratory tasks, this document can be considered a living document and reflects our current understanding of the protocol, as well as design decisions taken during implementation.

Besides collecting node-specific details in this document, we intend to contribute implementation-independent specifications to the cardano-blueprint initiative and also update the CIP-164 specification through pull requests as needed.

Document history

This document is a living artifact and will be updated as implementation progresses, new risks are identified, and validation results become available.

Version	Date	Changes
0.10	2026-06-19	Significantly extend technical design chapter
0.9	2026-03-19	Expand network sections
0.8	2026-03-02	Performance and quality assurance strategy
0.7	2026-02-13	Re-order technical design and add notes from prototyping
0.6	2025-11-25	Risks and mitigations with key threats
0.5	2025-10-29	Re-structure and start design chapter with impact analysis content
0.4	2025-10-27	Add overview chapter
0.3	2025-10-25	Add dependencies and interactions
0.2	2025-10-24	Add implementation plan
0.1	2025-10-15	Initial draft

2 Overview

Cardano as a cryptocurrency system fundamentally relies on an implementation of Ouroboros, the consensus protocol (TODO cite praos and genesis papers), to realize a permissionless, globally distributed ledger. The consensus protocol provides two essential properties that underpin Cardanos value proposition: **persistence** ensures immutability of confirmed transactions, while **liveness** guarantees that new valid transactions will be included. These properties enable secure and censorship-resistant transfer of value, as well as the execution of smart contracts in a trustless manner.

Ouroboros Leios introduces **high-throughput** as a third fundamental property, extending the currently deployed Ouroboros Praos variant. By enabling the network to process a significantly higher number of transactions per second, Leios addresses the economic scalability requirements

necessary to support a growing user base and application ecosystem. This enhancement transforms Cardano from a system optimized for security and decentralization into one that maintains these properties while achieving higher transaction processing capacity demanded by modern blockchain applications.

2.1 From research to implementation

As was the case for the Praos variant of Ouroboros, the specification embodied in the published and peer-reviewed research paper for Ouroboros Leios was not intended to be directly implementable. Initial research and development studies confirmed this expectation, identifying several unsolved problems with the fully concurrent block production design when considering the concrete Cardano ledger and what consequences this would have (TODO: cite suitable R&D reports, Tech Report #2, Impact analysis survey); further research is needed before those parts can be implemented.

The design presented in CIP-164, also known as Linear Leios, focuses on the core insight of utilizing the unused network bandwidth and computational resources during the necessary and eponymous calm periods of the Praos protocol. This approach provides an immediately implementable design that can deliver orders of magnitude higher throughput while preserving the security guarantees that make Cardano valuable.

The Linear Leios protocol operates by allowing a second, bigger type of block to be produced in the same block production opportunity. Block producers can produce and announce an endorser block (EB), which endorses additional transactions that would not fit within the Praos block. EBs are distributed through the network and subjected to validation by a committee of stake pools, who vote on their transaction data closures availability and validity. Only EBs that achieve a high threshold of stake-weighted votes become certified and can be included in the ledger through exclusive anchoring of a certificate in the subsequent block - now called a ranking block (RB). This mechanism allows for significantly higher transaction throughput while maintaining the security properties of the underlying Praos consensus. See the CIP for more details on the protocol specification and rationale itself.

2.2 Cardano node as a real-time system

The implementation of Leios must be understood in the context of the Cardano node as a concurrent, reactive system operating under real-time constraints in an adversarial environment. While real-time in this context does not refer to the millisecond-level hard deadlines found in industrial control systems, timely action at the scale of seconds nontheless remains crucial to protocol success and network security.

The currently deployed Praos implementation establishes clear data diffusion targets: blocks must reach 95% of nodes within the 5-second parameter, with target performance at 98% and stretch goals at 99%. While these are comfortably achieved most of the time, blocks are regularly adopted within 1 second across the network, there are some situations even in the current system

where the target is not reached. For example, due to reward calculations happening at the epoch boundary.

Despite being hard deadlines, these targets reflect the reality that network vulnerability increases when not being met. The protocols safety and liveness guarantees depend on honest nodes being able to propagate blocks rapidly enough to prevent adversarial forks from gaining traction. Failure to meet these timing constraints can lead to increased rates of short forks, reduced chain quality, and - if persistent - ultimately compromise the integrity of the ledger.

2.3 Concurrency and resource management

The current primary responsibilities of a Cardano node are roughly:

- Block diffusion: receiving chains from upstream, validate and select the best chain, and transmit chains downstream.
- Transaction submission: receiving, validating, and transmitting transactions to be included in blocks.
- Block production: creating new blocks and extending current chain when selected as slot leader.

Despite this apparent simplicity, this already results in a highly concurrent system once cardinalities of upstream and downstream network peers are considered. A `cardano-node` with the default configuration maintains 20 upstream hot peers, 10 upstream warm peers and can have up to a few hundred downstream connections, each of which may be simultaneously requesting or serving data. All of these operations share critical resources, including memory, CPU, and network bandwidth, requiring careful resource management to ensure timing requirements are met even under load.

Concretely, in the current system there are (including protocols for supporting features like peer sharing):

- 2 pipelined + 3 non-pipelined instances per upstream peers => 7 threads per upstream peer;
- 1 pipelined + 4 non-pipelined instances per downstream peer => 6 threads per downstream peer

Leios significantly expands this concurrency model by introducing new responsibilities:

- Endorser block and closure diffusion: receiving, validating, and transmitting EBs and their transaction closures.
- Voting and vote diffusion: receiving, validating, and transmitting own and foreign votes on EBs.

Given the proposed Leios mini-protocols, this would result in:

- 4 pipelined + 3 non-pipelined per upstream peers => 11 threads per upstream peer;

- 1 pipelined + 6 non-pipelined per downstream peer => 8 threads per downstream peer

With these two additional functionalities, each across many peers, the node set of concurrent tasks strictly increases. The implementation must ensure that the increased data flows and processing demands do not interfere with each other, or priorization mechanisms ensure to meet the stringent timing constraints necessary for protocol security.

2.4 Designing for the worst-case

Related to the principle of optimizing for the worst case, the security argument for Leios protocol depends critically on worst-case diffusion characteristics. Endorser blocks and their transaction closures must be small enough that the difference between optimistic diffusion (leading to successful certification) and worst-case diffusion remains bounded by the protocol parameter L_{diff} according to the protocols security argument.

If the optimistic, average-case performance is improved with suitable algorithms, data structures and optimizations, but the worst-case scenario is not, more conservative parameter choices would be required to maintain security guarantees. This would negate the anticipated benefits of the optimizations in the first place. Therefore, the implementation must prioritize ensuring that even in adverse network conditions or under attack, the diffusion of EBs and their closures remains within acceptable bounds.

> [!WARNING] TODO: the situation is not as dire though, we have some design freedom because strictly less work needs to be done on the worst-case path (e.g. rely on certified validity and cheaply build ledger states instead of validating transactions)

Besides, as with Praos, the enhanced information exchange requirements of Leios must not compromise the systems resilience against denial of service attacks and asymmetric resource consumption attempts. The implementation must maintain defensive properties while supporting the increased data flows and processing requirements that enable higher throughput.

2.5 Implementation imperatives

In summary, the technical design described in subsequent chapters must ensure that nodes continue to operate reactively and meet timing requirements despite increased responsibilities and data volumes. This requires careful bounding of resource usage and sophisticated prioritization mechanisms across concurrent responsibilities.

The complexity of this challenge emphasizes the critical importance of non-functional requirements specification for each component, rigorous performance engineering practices, and continuous benchmark validation throughout the development process. Only through systematic attention to these implementation details can the protocol deliver the security and performance properties that make Leios a valuable enhancement to Cardanos capabilities.

The following chapters detail the specific risks that inform architectural decisions, the concrete technical design that addresses these challenges, and the implementation plan that will deliver a production-ready system.

3 Implementation plan

The implementation of Ouroboros Leios represents a substantial evolution of the Cardano consensus protocol, introducing high throughput as a third key property alongside the existing persistence and liveness guarantees. The path from protocol specification to production deployment requires careful validation of assumptions, progressive refinement through multiple system readiness levels, and continuous demonstration of correctness and performance characteristics. This chapter outlines the strategy for maturing the Leios protocol design through systematic application of formal methods, simulation, prototyping, and testing techniques.

The result is an implementation plan that not only covers the path to active of CIP-164, but also serves as a rationale for what concrete steps will be taken on our product roadmap of realizing Ouroboros Leios.

> [!WARNING]
>
> TODO: mention on-disk storage and its availability; relevant for prototyping and early testnet (chain volume)
>
> TODO: incorporate or at least mention interactions with Peras
>
> TODO: also mention Genesis (potential to only do this later once testnet available?)

3.1 Approach

Research and development of distributed consensus protocols does not follow a linear waterfall process. Rather, the protocol design must be matured through various stages of validation, each building confidence in different aspects of the system. The peer-reviewed research paper provides strong theoretical guarantees under certain assumptions, but translating these guarantees into a working implementation that operates reliably on real-world infrastructure requires bridging substantial gaps. The implementation strategy must therefore balance multiple concerns: validating that core assumptions hold in practice, ensuring that refinements preserve essential properties, building developer confidence through rigorous testing, and ultimately securing acceptance from the governing bodies that must approve deployment to mainnet.

The challenge is compounded by the nature of the system itself. Cardano as deployed on mainnet is a globally distributed system with hard real-time constraints operating in an adversarial environment. Failures or performance degradation cannot be tolerated, as they directly impact the economic value and security guarantees that users depend upon. This necessitates an implementation approach that validates critical properties early, maintains continuous delivery of

working prototypes, and ensures transparency in both progress and limitations throughout the development process.

Three principles guide the implementation strategy: First, **early validation** of critical assumptions and risks enables discovery of fundamental problems as early as possible in the development cycle and reduces the likelihood for wasteful pivots and delays in delivery. Second, the implementation must progress through **continuous delivery** of increasingly capable prototypes rather than attempting to build the complete system in isolation. This allows for empirical validation at each stage and enables course corrections based on observed behavior. Third, **transparency** in both capabilities and limitations must be maintained throughout, ensuring that stakeholders including stake pool operators and delegated representatives can make informed decisions about deployment readiness.

These principles are also reflected in the choice of validation techniques applied at each stage. Formal methods provide the strongest guarantees of correctness but apply to abstracted models. Simulation enables exploration of protocol behavior under controlled conditions including adversarial models. Prototypes running on real infrastructure validate that theoretical performance bounds can be achieved in practice. Public testnets demonstrate end-to-end integration and allow the broader community to evaluate the system under realistic conditions.

3.2 Correctness in two dimensions

Formal specification and verification play a central role in ensuring correctness throughout the implementation process, which happens along two dimensions: - Maturity: Implementations maturing from proof of concept, prototype to production-ready release candidates - Diversity: Multiple emerging implementations of Cardano nodes using different programming langugages and targeting slightly different use cases

A protocol specification captured in a formal langugage like Agda, provides an unambiguous description of the protocol that can be checked for consistency and allows proving equivalence and other properties. A formal specification serves as the authoritative reference against which all implementations must be verified.

The approach to formal verification in Leios follows the trail of evidence methodology successfully applied in previous Ouroboros consensus implementations. Rather than attempting to verify the entire codebase directly, which becomes intractable for systems of this complexity, the methodology establishes correctness through a chain of increasingly refined specifications. For example, the high-level specification defines the protocol abstractly, while further refined specifications would focus on details such as message ordering and timing. Finally, an executable implementation is shown to correspond to the formal specification through a combination of techniques including type safety, property-based testing, and trace verification - often summarized as **conformance testing**.

Trace verification deserves particular attention as it provides a bridge between formal specifications and running code. The approach involves instrumenting both the formal specification and

the implementation to produce detailed execution traces. These traces can then be compared to verify that the implementation exhibits the same observable behavior as the specification for given inputs. For consensus protocols, the relevant observable behavior includes the sequence of blocks produced, the certificates generated, and the final ledger state. By systematically exploring the space of possible inputs including adversarial scenarios, high confidence can be achieved that the implementation faithfully realizes the specification.

Multiple implementations provide additional assurance through diversity. The primary Haskell implementation in `cardano-node` continues to serve as the reference, while alternative implementations in other languages are currently in development and will eventually increase the **node diversity** of Cardano. Alternative implementations on the node- or component-level serve multiple purposes: - validate that the specification is sufficiently precise and complete, - exercise different corner cases that might be missed by a single implementation, and - reduce the risk that a subtle bug in one implementation compromises the entire network.

The formal specification must be maintained as a living artifact throughout implementation. As design decisions are made to address practical concerns, these decisions must be reflected back into the specification to ensure it remains accurate. This bidirectional relationship between specification and other steps on the implementation plan is essential. The specification guides implementation, while implementation experience reveals necessary refinements to the specification. Documentation of these refinements and the rationale behind them provides crucial context for future maintainers and for external review. Consequently, the specification itself and other implementation-independent artifacts will be contributed to the `cardano-blueprint` initiative.

3.3 Simulation and protocol validation

Simulations provide a very controlled environment for exploring protocol behavior before deploying to real infrastructure. Two complementary simulation approaches have been used so far to validate the proposed protocol in CIP-164, each with distinct strengths and even using different implementation languages.

A discrete event simulation implemented in Rust, models Leios message exchanges between nodes, abstracting lower-level details for speedrunning orders of magnitude faster than real time to enable statistical analysis over thousands of runs with complete observability and arbitrary adversarial behavior injection. This validates security arguments by systematically exploring protocol behavior under varying loads, expected data diffusion in small to medium sized network topologies, or adversarial scenarios like data withholding, and exploration of protocol parameters before testnet deployment.

Another Haskell-based simulation using IOSim and the actual network framework used in the `cardano-node`. This reduces model-implementation divergence while enabling studies of the dynamic behavior and resource management in detail. While IOSim is used in the existing network and consensus layers through property-based testing, and extends naturally to Leios components, the simulator built from this was not able to scale to large networks.

Both approaches necessarily abstract real system details and thus provide evidence of correct behavior under idealized conditions and suggest workable parameters, but cannot definitively predict real-world performance. Maintaining simulation synchronization with evolving implementation requires discipline, but enables rapid exploration of alternatives, early feature validation, and serves as executable documentation for new developers.

3.4 Prototyping and adversarial testing

Prototypes on real infrastructure validate performance characteristics that simulation typically cannot guarantee. The line between simulation and prototyping is blurry, but both concepts share the trait of allowing rapid exploration of the most uncertain aspects of the design before committing to a full implementation. Referring back to the key threats and assumptions to validate early, the primary focus of prototyping is on network diffusion performance under high throughput conditions and adversarial scenarios.

Network diffusion prototype: An early implementation of the actual Leios network protocols and freshest-first delivery mechanisms, that allows running experiments with various network topologies. Ledger validation of Leios concepts is stubbed out and transmitted data is generated synthetically to focus purely on network performance. Deployed to controlled environments like local devnets and private testnets like the the Performance and Testing cluster, this prototype systematically explores how performance scales with network size and block size, tests different topologies, and crucially answers whether the real network stack achieves the diffusion deadlines required by protocol security arguments. Key measurements include endorser block arrival time distributions, freshest-first multiplexing effectiveness, topology impact on diffusion, and behavior under adversarial scenarios including eclipse attempts and targeted withholding. These measurements will answer questions like, how much freshest-first delivery we need, whether the proposed network protocols are practical to implement and what protocol parameter are feasible.

Adversarial testing represents a crucial aspect of prototype validation. In a controlled environment, some nodes can still be configured to exhibit adversarial behaviors such as sending invalid blocks, withholding information, or attempting to exhaust resources of honest nodes. Observing how honest nodes respond provides evidence that the mitigations described in the design are effective. Despite using real network communication, such systems can still be determinstically simulation-tested using tools like Antithesis, which is currently picked up also by node-level tests in the Cardano community via moog. If we can put this technique to use for adversarial testing of Leios prototypes and release candidates, this can greatly enhance our ability to validate the protocol under challenging conditions by exploring a much wider range of adversarial scenarios than would be feasible through manually created rigit test scenarios.

Beyond networking prototypes, additional focused prototypes may be created to address other known unknowns of the implementation:

Ledger validation benchmark: measures the throughput of transaction validation and ledger state updates. This is critical for understanding whether a node can process the contents of

large endorser blocks within the available time budget and confirm whether our models for transaction validation are correct. The benchmark explores different transaction types and sizes, measures the impact of caching strategies, and validates the performance improvement from the no-validation application of certified transactions.

Cryptographic primitives prototype: validates the performance and correctness of the BLS signature scheme integration. This includes key generation, signing, verification, and aggregation operations. The prototype must demonstrate that batch verification of large numbers of votes can complete within the voting period deadline. It also serves to validate the proof-of-possession mechanism and explore key rotation techniques.

Focused prototypes provide empirical data that complements the theoretical analysis. They reveal where optimizations are necessary and validate that the required performance is achievable with available hardware. They also serve to build developer confidence in the feasibility of the overall design, as well as directly validate and inform architectural decisions. Discovering a fundamental performance limitation early, through prototyping, is far preferable to discovering it late during testnet deployment or, worse, after mainnet deployment.

3.5 Public testnets and integration

A public testnet serves distinct purposes over simulations and controlled environments: it requires integration of all components into a complete implementation, enables for tests under realistic conditions with diverse node operators and hardware, and allows the community to experience enhanced throughput directly. While some shortcuts can still be made, the testnet-ready implementation must offer complete Leios functionality - endorser block production and diffusion, vote aggregation, certificate formation, ledger integration, enhanced mempool - plus sufficient robustness for continuous operation and operational tooling for deployment and monitoring.

The testnet enables multiple validation categories. Functional testing verifies correct protocol operation: nodes produce endorser blocks when elected, votes aggregate into certificates, certified blocks incorporate into the ledger, and ledger state remains consistent. Performance testing measures achieved throughput against business requirements - sustained transaction rate, mempool-to-ledger latency, and behavior under bursty synthetic workloads. Adversarial testing is limited on a public testnet, but some attempts with deliberately misbehaving nodes can be made on withholding blocks, sending invalid data, attempting network partitioning, or resource exhaustion.

The testnet integrates ecosystem tooling: wallets handling increased throughput, block explorers understanding new structures, monitoring systems tracking Leios metrics, and stake pool operator documentation and deployment guides. Crucially, the testnet further enables empirical parameter selection (size limits, timing parameters), where simulation provides initial guidance but real-world testing with community feedback informs acceptable mainnet values.

Software deployed to the public testnet progressively converges toward mainnet release candidates. Early deployments may use instrumented prototypes lacking production optimizations; later upgrades run increasingly complete and optimized implementations. Eventually, all changes as outlined in this design document must be realized in the `cardano-node` and other node implementations. This progressive refinement maintains community engagement while preserving engineering velocity. Traces from testnet nodes can still be verified against formal specifications using the trace verification approach, ultimately linking the abstraction layers.

3.6 Mainnet deployment readiness

Mainnet deployment requires governance approval and operational readiness beyond technical validation. The Cardano governance process involves delegated representatives and stake pool operators who need clear understanding of proposed changes, benefits, and risks. Technical validation evidence from formal methods, simulation, prototyping, and testnet operation must be communicated accessibly beyond technical documentation.

Operational readiness encompasses stake pool operator testing in their environments, updated procedures and training, clearly documented upgrade procedures, updated monitoring and alerting systems, and prepared support channels. The hard fork combinator enables relatively smooth transitions, but Leios represents substantial consensus changes. Conservative timeline estimates must account for discovering and addressing unexpected issues - a normal part of the hard-fork scheduling process. The months of validation and refinement required before prudent mainnet deployment reflect the critical nature of modifications to a system holding substantial economic value and providing essential services that users depend upon.

> [!WARNING]
>
> TODO: more thoughts:
>
> - why (deltaq) modeling? quick results and continued utility in parameterization
> - parameterization in general as a (communication) tool; see also Peras parameterization dashboard https://github.com/tweag/cardano-peras/issues/54
> - whats left for the hard-fork after all this? more-and-more testing / maturing, governance-related topics (new protocol parameters, hard-fork coordination)

4 Dependencies and interactions

The changes necessary to realizing Leios must integrate carefully with existing infrastructure and emerging features. This section examines the critical dependencies that must be satisfied before Leios deployment, identifies synergies with parallel developments, and analyzes potential conflicts that require careful coordination. The analysis informs both the implementation timeline and architectural decisions throughout the development process.

4.1 On-disk storage of ledger state

> [!WARNING]
>
> TODO: Add some links and references to UTxO-HD and Ledger-HD specification and status

The transition from memory-based to disk-based ledger state storage represents a fundamental prerequisite for Leios deployment. This dependency stems directly from the throughput characteristics that Leios is designed to enable.

At the time of writing, the latest released `cardano-node` implementation supports UTxO state storage on disk through UTxO-HD, while other parts of the ledger state including reward accounts are put on disk within the Ledger-HD initiative. The completion of this transition is essential for Leios viability, as the increased transaction volume would otherwise quickly exhaust available memory resources on realistic hardware configurations.

The shift to disk-based storage fundamentally alters the resource profile of node operation. Memory requirements become more predictable and bounded, while disk I/O bandwidth and storage capacity emerge as primary constraints. Most significantly, ledger state access latency necessarily increases relative to memory-based operations, and this latency must be accounted for in the timing constraints that govern transaction validation.

Early validation through comprehensive benchmarking becomes crucial to identify required optimizations in ledger state access patterns. The (re-)validation of orders of magnitude bigger transaction closures, potentially initiated by multiple concurrent threads, places particular stress on the storage subsystem, as multiple validation threads may contend for access to the same underlying state. The timing requirements for vote production - where nodes must complete endorser block validation within the L_{vote} period - on one hand, and applying (without validation) thousands transactions during block diffusion on the other hand, impose hard constraints on acceptable access latencies.

These performance characteristics must be validated empirically rather than estimated theoretically. The ledger prototyping described in the implementation plan must therefore include realistic disk-based storage configurations that mirror the expected deployment environment.

4.2 Synergies with Peras

The relationship between Ouroboros Leios and Ouroboros Peras presents both opportunities for synergy and challenges requiring careful coordination. As characterized in the Peras design document, the two protocols are orthogonal in their fundamental mechanisms, Leios addressing throughput while Peras improves finality, but their concurrent development and potential deployment creates several interaction points.

Resource contention and prioritization emerges as the most immediate coordination challenge. Both protocols introduce additional network traffic that competes with existing Praos

communication. The resource management requirements for Leios - prioritizing Praos traffic above fresh Leios traffic above stale Leios traffic - must be extended to also accommodate Peras network messages. Any prioritization scheme requires careful analysis of the timing constraints for each protocol to ensure that neither compromises the others security guarantees. Current understanding is that Peras traffic should be prioritized above both, stale and fresh Leios traffic, such that Leios protocol burst attacks may not force Peras into a cooldown period.

Vote diffusion protocols present a potential area for code reuse, though this opportunity comes with important caveats. The Leios implementation will initially evaluate the vote diffusion protocols specified in CIP-164 for their resilience against protocol burst attacks and general performance characteristics. Once the Peras object diffusion mini-protocol becomes available, it should also be evaluated for applicability to Leios vote diffusion. However, the distinct performance requirements and timing constraints of the two protocols may ultimately demand separate implementations despite structural similarities.

Cryptographic infrastructure offers the most promising near-term synergy. Both protocols are based on signature schemes using BLS12-381 keys, creating an opportunity for shared cryptographic infrastructure. If key material can be shared across protocols, stake pool operators would need to generate and register only one additional key pair rather than separate keys for each protocol. This shared approach would significantly simplify the bootstrapping process for whichever protocol deploys second.

The Peras requirement for forward secrecy may necessitate the use of Pixel signatures on top of the BLS12-381 curve, in addition to BLS (as a VRF) for committee membership proofs, but this is completely independent of Leios requirements. Furthermore, the proof-of-possession mechanisms required for BLS aggregation are identical across both protocols, allowing for shared implementation and validation procedures.

Protocol-level interactions between Leios certified endorser blocks and Peras boosted blocks represent a longer-term research opportunity. In principle, the vote aggregation mechanisms used for endorser block certification could potentially be leveraged for Peras boosting, creating a unified voting infrastructure. However, such integration is likely undesirable for initial deployments due to the complexity it would introduce and the dependency it would create between the two protocols. In the medium to long term, exploring these interactions could yield further improvements to both throughput and finality properties.

4.3 Era and hard-fork coordination

As already identified in the impact analysis, Leios requires a new ledger era to accommodate the modified block structure and validation rules. The timing of this transition must be carefully coordinated with the broader Cardano hard-fork schedule and other planned protocol upgrades.

At the time of writing, the currently deployed era is `Conway`, with `Dijkstra` planned as the immediate successor. Current plans for `Dijkstra` include nested transactions and potentially Peras

integration. Before that, an intra-era hard fork is planned for early 2026 to enable additional features within the `Conway` era still.

A new era is always required when the allowed encoding of block bodies and transactions change. As `Dijkstra` is the current staging era, it will also be the integration point for Leios-specific format changes. Should development timelines turn out not to align with the inter-era hard-fork schedule to `Dijkstra`, there are two options:

- Postpone Leios deployment until after `Dijkstra`, moving Leios block format changes into the subsequent era `Euler`.
- Leios block format encoding specification and implementation remains in `Dijkstra`, but ledger validation is always failing until an intra-era hard-fork enables it.

While the first option appears cleaner, it could introduce substantial delays depending on the community-agreed pace on new era definition and deployments. The second option on the other hand requires definite understanding on the serialization format ahead of time, where any further change would result in option one of targeting `Euler`, but with the added friction of feature-flagging Leios functionality before its moved to `Euler` - the worst of both options.

Deploying both Peras and Leios within the same hard fork is technically possible but increases deployment risk. Both protocols represent significant consensus changes that affect network communication patterns, resource utilization, and operational procedures. The complexity of coordinating these changes, validating their interactions, and managing the upgrade process across the diverse Cardano ecosystem suggests that sequential deployment provides a more conservative and manageable approach. Both options above would allow for that via two subsequent protocol versions, but also both in one hard-fork if the risk is deemed acceptable.

4.4 Interactions with Genesis

Ouroboros Genesis enables nodes to bootstrap safely from the genesis block with minimal trust assumptions, completing the decentralization of Cardanos physical network infrastructure. Genesis integration with Leios requires **no** changes to the existing Genesis State Machine, though practical considerations on synchronization remain important.

Genesis compatibility directly follows from the protocol design. The Genesis protocol operates on ranking block headers for chain density calculations and bootstrapping decisions. Since Leios preserves the existing ranking block sequence while only adding certificates of endorser blocks, the fundamental Genesis mechanisms remain unchanged. No modifications to the Genesis State Machine are expected, as it continues to evaluate the unchanged chain growth.

Chain synchronization in general becomes more complex under Leios due to the multi-layered block structure. Syncing nodes must fetch both ranking blocks and their associated certified endorser blocks to construct a complete view of the chain. A node that downloads only ranking blocks cannot reconstruct the complete ledger state, as the actual transactions content resides within closures of the endorser blocks referenced by certificates on the ranking blocks. The `LeiosFetch` mini-protocol addresses this requirement through the `MsgLeiosBlockRangeRequest`

message type, enabling efficient batch fetching of complete block ranges during synchronization. This allows nodes to request not only a range of ranking blocks but also all associated endorser blocks and their transaction closures in coordinated requests. Parallel fetching from multiple peers becomes critical for synchronization performance, as the data volume substantially exceeds that of traditional Praos blocks.

> ![WARNING]
>
> TODO: Chain synchronization / syncing node discussion could be moved to the respective section in the architecture/changes chapter

4.5 Impact on Mithril

While Mithril operates as a separate layer above the consensus protocol and does not directly interact with Leios mechanisms, the integration requires consideration of several practical compatibility aspects.

The most prominent feature of Mithril is that it serves verifable snapshots of the `cardano-node` databases. The additional data structures introduced by Leios (e.g. the EBStore) must be incorporated into the snapshots that Mithril produces and delivers to its users. Beyond that, Mithril needs to also extend its procedures for digesting and verifying the more complex chain structure including endorser blocks and their certification status.

Mithril relies on a consistent view of the blockchain across all participating signers. Hence, the client APIs used by Mithril signers may require updates depending on which interfaces are utilized. While Mithril initially focused on digesting and signing the immutable database as persisted on disk, the consideration of using `LocalChainSync` for signing block ranges introduces potential interaction points with Leios induced changes to client interface.

In summary, Leios will not require fundamental changes to Mithrils architecture but requires careful attention to data completeness and consistency checks in the snapshot generation and verification processes.

5 Risks and mitigations

This chapter bridges the implementation plan with concrete technical design by examining selected threats that directly inform architectural decisions and validation priorities. The focus is on implementation-specific risks rather than general protocol threats, which are either mitigated by design or othterwise covered in the threat model.

The threats examined here represent scenarios that could compromise the implementations ability to deliver Leios promised benefits while maintaining Praos security guarantees. Each threat analysis motivates specific technical requirements, validation experiments, or design constraints that shape the implementation outlined in subsequent chapters.

5.1 Key threats

The following threats have been selected for detailed analysis based on their potential to inform critical implementation decisions. These represent attack vectors that emerged prominently during research, have significant implications for system performance under adversarial conditions, or require empirical validation through prototyping and testing.

5.1.1 Data withholding

In a data withholding attack (**ATK-LeiosDataWithholding**, see also threat vectors #20, #21 and #22), the adversary deliberately prevents the diffusion of endorser block transaction closures to disrupt certification and degrade network throughput. This attack exploits the fundamental dependency between transaction availability and EB certification, targeting the gap between optimistic and worst-case diffusion scenarios that underlies Leios security argument.

The attack operates by manipulating timing and availability of transaction data required for EB validation. When an EB is announced via an RB header, voting committee members must acquire and validate the complete transaction closure before casting votes. The adversary can exploit this in several ways: withholding the EB body itself, selectively withholding individual transactions, or strategically timing data release to exceed the L_{vote} deadline.

Direct threshold impact. The most direct form involves an adversarial block producer creating valid EBs but refusing to serve transaction closures when requested by voting nodes. Since committee members cannot validate unavailable transactions, they cannot vote for certification, effectively nullifying the EBs throughput contribution. More sophisticated variants involve network-level manipulation where the adversary controls network relays to selectively prevent transaction propagation to specific voting committee members.

Consider an adversary controlling 15% of stake attempting to prevent honest EBs from achieving the 75% certification threshold. The adversary must withhold transaction data from enough voting committee members to reduce available honest stake below 75%. Since the adversary controls 15% stake directly, they need to prevent an additional 10% of honest stake from voting. This demonstrates how modest adversarial stake combined with strategic network positioning could significantly impact honest EB certification.

Attack on safety. While throughput degradation represents the obvious impact, the most dangerous variant targets blockchain safety itself. The adversary can strategically delay transaction data release to create scenarios where EBs achieve certification but cannot be processed by honest nodes within the required timeframe. Just before the voting deadline, they release data to a subset of voting committee members (who are approximately colocated in the worst case)enough to achieve certification, but not to all network participants. The resulting certificate gets included in a subsequent RB, but honest block producers cannot acquire the certified EBs transaction closure within L_{diff}.

By reducing the number of honest nodes that received the EB data in time for certification, the adversary also impairs subsequent diffusion. With fewer nodes initially possessing the complete transaction closure, propagation becomes slower and less reliable, potentially extending diffusion times beyond protocol anticipation. This would represent a violation of Praos timing assumptions. While missing the deadline occasionally does not break safety, short forks are normal in Ouroboros, persistent violations can lead to longer forks and degraded chain quality. In summary, the attack fundamentally challenges the security arguments assumption that the difference between optimistic and worst-case diffusion remains bounded by L_{diff}.

Mitigation relies primarily on the protocol design ensuring that diffusion timing remains bounded even under adversarial conditions. The certification mechanism provides defense against stake-based withholding by requiring broad consensus before including EBs in the ledger. Network-level attacks require sophisticated countermeasures including redundant peer connections, time-outs that punish non-responsive nodes, and strategic committee selection considering network topology.

The implementation must validate empirically that real-world network conditions support the timing assumptions underlying the security argument through adversarial diffusion testing.

5.1.2 Protocol bursts

In a protocol burst attack (**ATK-LeiosProtocolBurst**, see also threat vector #23) the adversary withholds a large number of EBs and/or their closures over a significant duration and then releases them all at once. This will lead to a sustained maximal load on the honest network for a smaller but still significant duration, a.k.a. a burst. The potential magnitude of that burst will depend on various factors, including at least the adversarys portion of stake, but the worst-case is more than a gigabyte of download. The cost to the victim is merely the work to acquire the closures and to check the hashes of the received EB bodies and transaction bodies. In particular, at most one of the EBs in the burst could extend the tip of a victim nodes current selection, and so thats the only EB the victim would attempt to fully parse and validate. Even without honest nodes needing to validate the vast majority of the burst, the sheer amount of concentrated bandwidth utilization can be problematic.

Attack magnitude. Suppose the adversary controls 1/3rd stake and theyre issuing nominal RBs. Recall that CIP-164 requires each honest node to attempt to acquire any EB that was promptly announced within the last 12 hours. Even if its too late for the honest node itself to vote on a tardy EB, the lack of global objective information implies the node must not assume the EB cannot be certified by other nodes in the network. Thus, the honest node might later need to switch to a fork that requires having this EB, and that switch ideally wouldnt be delayed by waiting on that EBs arrival; the node should still acquire the EB as soon as it can. For this attack, the adversary would announce each EB promptly (by diffusing the corresponding RB headers on-time), but withhold the mini protocol messages that actually initiate the diffusion of substantial Leios traffic throughout the honest network. Only after withholding every EBs

diffusion for 12 hours would they suddenly release them. In this scenario, which is not the worst-case, the average would be approximately 2160 * (1/3) = 720 EBs, but there could be hundreds more merely due to luck and multi-leader slots. There could be several thousand if the adversary is also grinding, for example, and/or had closer to 50% stake, etc. If each of the attackers EBs has the maximum size of 500 kilobytes of tx references and 12.5 megabytes of actual txs, which dont even need to be valid, then thats an average of 720 * (12.5 + 0.5 megabytes) = 9.36 gigabytes the honest nodes will be eagerly diffusing throughout the network.

Resource contention. For however long it takes for the network to (carefully) diffuse 10 gigabytes, honest traffic might diffuse more poorly. CIP-164 requires that Praos traffic will be preferred over Leios traffic and that fresher Leios traffic will be preferred over stale Leios traffic. That would prevent the burst from degrading contemporary honest traffic if the prioritization could be perfect.

However, there are some infrastructural resources that cannot be prioritized perfectly nor instantly reapportioned, including: CPU, memory, disk, disk bandwidth, and buffer utilization on the nodes themselves but also along the Internet routers carrying packets between Cardano peers. One non-obvious concern is that cloud providers often throttle users exhibiting large bursts of bandwidth, so a node might perform fine outside of a protocol burst but struggle disproportionately during one. A node in a data center might not struggle at all to diffuse the 10 gigabytes over the course of each 12 hours but be very slow to diffuse it in a single burst that arrives every 12 hours.

Some of CPU and memory costs will scale in the number of txs rather than the number of EBs, which can be a ratio of more than 10,000 to 1. If none of the 720 EBs overlap, then there would be more than 7,200,000 unique txs on average that the honest nodes need to keep track of during this burst. The identifying hashes of those txs alone is more than 230 megabytes. To maximize the bookkeeping overhead, for example, the adversary might choose to diffuse all of the EB bodies before diffusing any of their closures.

It remains an engineering challenge to achieve as much prioritization as possible, especially during a protocol burst, without unnecessarily delaying the diffusion of any EB. That is, determining how to prevent this kind of protocol burst from increasing the latency of contemporary Praos and Leios traffic among honest nodes.

The adversary is only able to issue EBs at an average rate in proportion to their resources (stake and grinding). There will be some variance, but in general they can do smaller bursts more often or larger bursts less often. However, the Praos security arguments parameters represent the worst-case, so the largest burst fundamentally challenges the current Praos security argument even if it can only happen rarely to whatever extent the prioritization schemes of CIP-164 are imperfectly implemented.

5.2 Assumptions to validate early

Following the principle of early validation outlined in the implementation plan, several critical assumptions underlying Leios security argument must be validated before we can commit to full scale implementation and deployment. These assumptions represent potential failure points where theoretical models may not match real-world performance.

- **Worst case diffusion of EBs given certain honest stake is realistic.** The security argument assumes that even under adversarial conditions, EBs can be diffused to honest nodes within bounded timeframes. This assumption must be validated under various network topologies and attack scenarios to ensure the L_{diff} parameter provides adequate protection.
- **The Cardano network stack can realize freshest-first delivery sufficiently well.** Prioritizing Praos, over recent Leios , over stale Leios traffic is essential for mitigating protocol burst attacks. Real-world validation must demonstrate that the network layer can maintain this prioritization under load without significantly impacting Praos traffic.
- **A real ledger can process orders of magnitude higher transaction loads as expected.** Leios assumes that nodes can validate and apply large transaction sets within tight timing constraints. This requires empirical validation of transaction validation throughput, especially when combined with disk-based ledger storage and concurrent processing demands.

The prototyping and adversarial testing phase of the implementation plan is specifically designed to validate these assumptions through controlled experiments. Only with such validation we can confidently design and implement the components that realize a Leios consensus.

6 Technical design

> [!CAUTION]
>
> FIXME: The next few sections are basically the relevant parts of the impact analysis and ought to be expanded with concrete implementation designs.
>
> When transferring things from impact analysis, the **REQ-** requirements as well as the **NEW-..** and **UPD-..** references were kept. Not sure if we need all of them as references between concepts and designs.
>
> [!WARNING]
>
> TODO: Did we cover everything? The following mind map provides an overview of Leios-related changes across components:

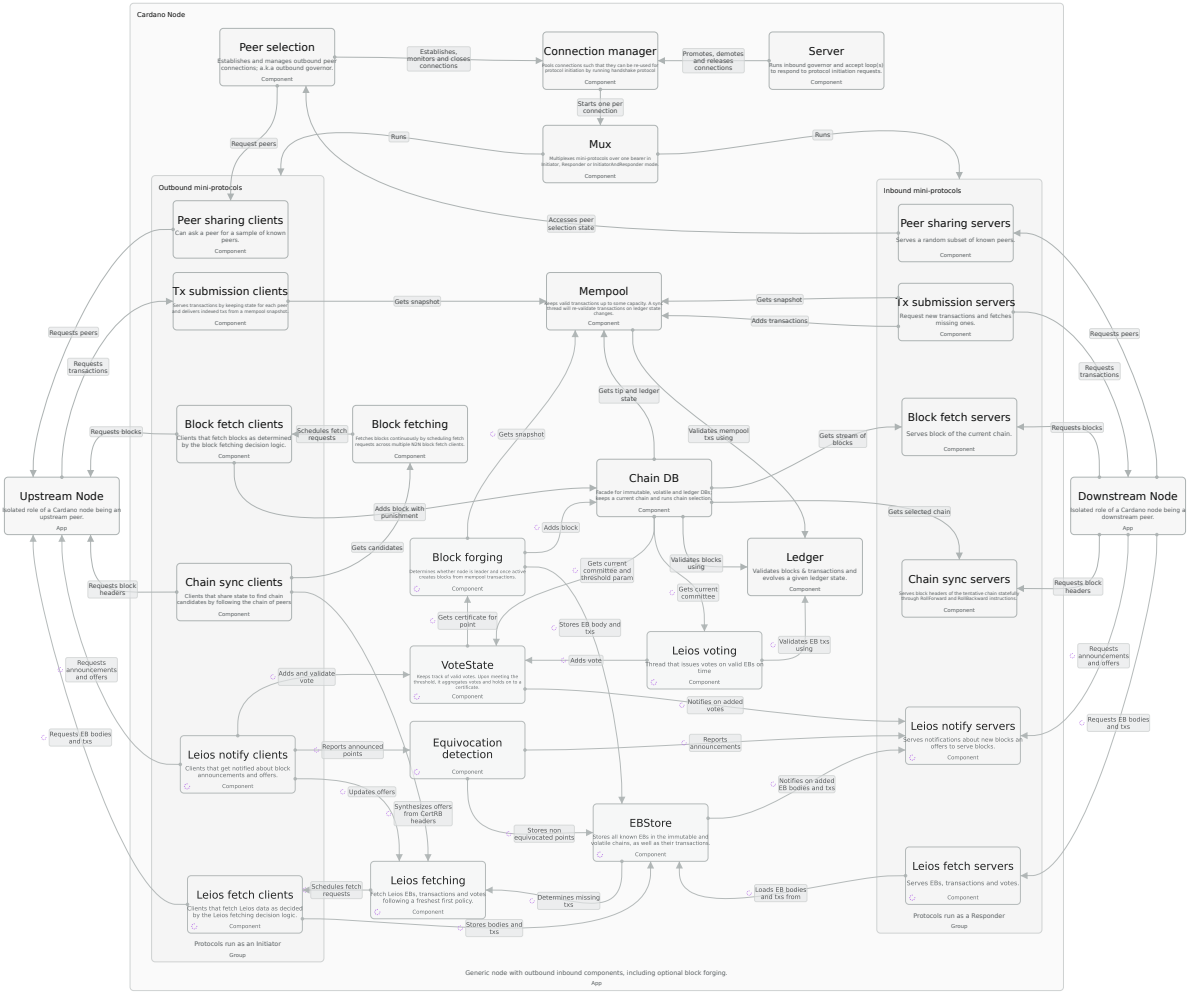
```
mindmap
  root((Leios tasks, core devs))
    ((Ledger))
      Serialization
        Certs in RB bodies<br>- akin to Peras
        Cert codecs/CDDL
      Cert validation
      Voting key registration/rotation via tx certs<br>- akin to Peras
      Commitee selection
      Registered keys / selected committee queries
      New protocol parameters
    ((Consensus))
      Serialization
        New fields in RB header
        EB codecs/CDDL
        Vote codecs/CDDL
      Storage
        EBs - imm and vol
        Txs of EBs - imm and vol
        Votes - only vol (or only in-mem)
        Tx cache
      High-throughput mempool
      Forging updates
      Vote creation / voting
      Vote validation
      Certifcate / vote aggregation
      NTC chain sync server inlined blocks
      New LocalStateQuery queries
      Prioritize Praos threads?
      Genesis State Machine transition predicates?
    ((Network))
      Prioritize Praos traffic
      Prioritize Praos threads
    ((Network&Consensus))
      New mini protocols
        Message codecs/CDDL
        Tune size and time limits
        Tune pipelining depth
      EB fetching logic
        Freshest first / oldest first policy
          either: conservative pipelining depths
          and/or else: server-side reordering
      Vote fetching logic
```

```
((Node&CLI))
  Feature flags for dev phases
  New CLI queries
  Key registration/rotation
  New config data
```

6.1 Architecture

While being a significant change to the consensus protocol, Leios does not require fundamental changes to the overall architecture of the `cardano-node`. Several new components will be needed for the new responsibilities related to producing and relaying Endorser Blocks (EBs) and voting on them, as well as changes to existing components to support higher throughput and freshest-first-delivery. The following diagram illustrates the key components where new and updated components are marked in purple:

> [!WARNING]
>
> TODO: Should consider adding Leios prefixes to VoteStore (to not confuse with PerasVoteDB), i.e. LeiosVoteDB?



[!WARNING]

TODO: Give an overview of the diagram and link to later sections for details

6.1.1 Design guidelines

The guidelines below inform how the new components in the diagram are structured internally and how they integrate with the existing consensus codebase. They are not Leios-specific, but articulate a discipline that consensus extensions in general could follow.

6.1.1.1 Avoid conditional wiring Wire Leios components up unconditionally and let the ledger drive their behavior. The ledger is the single source of truth for which features are active, which parameters apply, and who participates; code should not carry a parallel opinion. A Leios-capable node running in a pre-Leios era constructs every state store, runs every thread, and registers every mini-protocol exactly as it would post-fork. Those components consult the ledger, find no committee or no eligible round, and produce no work, no traffic, and no state changes. When the ledger transitions to Leios at an epoch boundary, the same code begins producing

effects, because its ledger queries now return data. Nothing switches on; nothing is spawned for the first time. The canonical idiom is a ledger query whose result drives behavior — for example a `getCurrentCommittee :: LedgerState blk -> Maybe Committee` returning `Nothing` pre-Leios, with all consumers short-circuiting on `Nothing`. The era logic is encoded once, in the ledger; consuming code is symmetric across eras.

Dont gate thread spawning, mini-protocol registration, or state-store creation on era status; wire all three up unconditionally at startup, even if their backing components have nothing to do until the ledger says otherwise. And dont dispatch runtime behavior between old and new code paths based on era — there is one behavior, and the ledger decides what it produces.

The trade-off is a steady-state cost: idle threads parked on STM, empty caches, and registered-but-quiet channels on every long-lived peer connection — paid universally, including by nodes that sync only historical chains and will never see Leios active. We accept it in exchange for a single source of truth and a hard fork that is purely a ledger event, not a code path. The Hard Fork Combinator already requires us to keep present-but-inactive behavior for past and future eras, and the guideline leans into that: Leios is enabled by entering the Leios ledger era and by the corresponding protocol parameters — set through genesis files or on-chain governance like any other parameter — not by a separate runtime feature flag. The one exception is NodeToNode protocol versioning, which is genuinely a code-side decision: shipping a new mini-protocol requires a code release, so the version that introduces the Leios mini-protocols is negotiated at handshake rather than read from the ledger. The guideline still rejects per-component, per-thread, per-protocol branching that would recreate feature awareness throughout the codebase.

6.1.1.2 Be as concrete as possible Use concrete types mono-morphic types where possible; only add a parameter when todays code already needs the alternative. The clearest established example is `m`: consensus code is parametric over `m` because `IO` runs the production node and `IOSim` runs deterministic tests — both committed inhabitants, exercised on every CI run. Everything else stays concrete by default: `LeiosVote` is a BLS-signed record, `Committee`, `Round`, and `EB` are concrete data types, and component handles like `LeiosVoteState m` and `LeiosDbHandle m` are parametric only over `m`. Where consensus already imposes polymorphism — the `blk` block format or `LedgerState blk` at the ledger boundary — a single typeclass or record of accessors consumes `blk` and projects to those concrete types, so the polymorphism does not leak further. When a future consumer is committed (for example, tests that need a fast deterministic signature scheme rather than running real BLS under IOSim), the corresponding parameter is added at the point the second inhabitant exists, not in anticipation.

Dont add parameters for hypothetical consumers: no open type families or class methods for unspecified future formats, no `crypto` or `era` parameters propagated through Leios types without a test or boundary that actually uses them. And dont let `blk` leak past the `LedgerState blk` boundary into Leios-internal types; `LeiosVoteState m` and `LeiosDbHandle m` are the ceiling for component handle parameterization.

The trade-off mirrors structural inertness: we accept a refactor cost later if the protocol generalizes in exchange for fewer type parameters and constraints to read today. Refactoring concrete

code is mechanical work that the type checker guides, while every parameter used or not appears at every site that mentions the type and shapes how the code reads. This guideline applies to new Leios code; existing consensus interfaces stay as they are. The line between present need and anticipated need is judgment, not a rule; a present need six weeks away is still present. When introducing polymorphism, the question to answer in the PR description is *which current consumer needs the alternative inhabitant?* If the answer is none yet, the answer is delete the parameter.

6.2 Resource management

The protocol requires resource-management to prioritize Praos traffic and computation over all Leios traffic, and prioritize younger EBs over older ones:

- **REQ-PrioritizePraosOverLeios** The node must prioritize Praos traffic and computation over all Leios traffic and computation so that the diffusion and adoption of any RB is only negligibly slower.
- **REQ-PrioritizeFreshOverStaleLeios** The node must prioritize Leios traffic and computation for younger EBs over older EBs (a.k.a. freshest first delivery).

These requirements can be summarized as: Praos > fresh Leios > stale Leios. The Consensus layer implements the scheduling logic to satisfy these requirements, while the Network layer (see below) implements the protocol mechanisms. Looking forward, Peras should also be prioritized over Leios, since a single Peras failure is more disruptive to Praos than a single Leios failure.

The fundamental idea behind Leios has always been that the Praos protocol is inherently and necessarily bursty. Leios should be able to freely utilize the nodes resources whenever Praos is not utilizing them, which directly motivates **REQ-PrioritizePraosOverLeios**. It is ultimately impossible to achieve such time-multiplexing perfectly, due to the various latencies and hystereses inherent to the commodity infrastructure (non real-time operating systems, public Internet, etc). On the other hand, it is also ultimately unnecessary to time-multiplex Praos and Leios perfectly, but which degree of imperfection is tolerable? See ATK-LeiosProtocolBurst for an example scenario in which resources are most competed for that motivates the technical design.

Contention for the following primary node resources might unacceptably degrade the time-multiplexing between Praos and Leios:

- **RSK-LeiosPraosContentionNetworkBandwidth** This is not anticipated to be a challenge, because time-multiplexing the bandwidth is relatively easy. In fact, Leios traffic while Praos is idle could potentially even prevent the TCP Receive Window from contracting, thus avoiding a slow start when Praos traffic resumes.
- **RSK-LeiosPraosContentionCPU** This is not anticipated to be a challenge, because todays Praos node does not exhibit major CPU load on multi-core machines. Leios might have more power-to-weight ratio for parallelizing its most expensive task (EB validation), but that parallelization isnt yet obviously necessary. Thus, even Praos and Leios together do not obviously require careful orchestration on a machine with several cores.

- **RSK-LeiosPraosContentionGC** It is not obvious how to separate Praos and Leios into separate OS processes, since the ledger state is expensive to maintain and both protocols frequently read and update it. When the Praos and Leios components both run within the same operating system process, they share a single instance of the GHC Runtime System (RTS), including eg thread scheduling and heap allocation. The sharing of the heap in particular could result in contention, especially during an ATK-LeiosProtocolBurst (at least the transaction cache will be doing tens of thousands of allocations, in the worst-case). Even if the thread scheduler could perfectly avoid delaying Praos threads, Leios work could still disrupt Praos work, because some RTS components exhibit hysteresis, including the heap.
- **RSK-LeiosPraosContentionDiskBandwidth** Praos and Leios components might contend for disk bandwidth. In particular, during a worst-case ATK-LeiosProtocolBurst, the Leios components would be writing more than a gigabyte to disk as quickly as the network is able to acquire the bytes (from multiple peers in parallel). Praoss disk bandwidth utilization depends on the leader schedule, fork depth, etc, as well as whether the node is using a non-memory backend for ledger storage (aka UTxO HD or Ledger HD). For non-memory backends, the ledgers disk bandwidth varies drastically depending on the details of the transactions being validated and/or applied: a few bytes of transaction could require thousands of bytes of disk reads/writes.
  - Note that the fundamental goals of Leios will imply a significant increase in the size of the UTxO. In response, SPOs might prefer enabling UTxO HD/Ledger HD over buying more RAM.

Both GC pressure and disk bandwidth are notoriously difficult to model and so were not amenable to the simulations that drove the first version of the CIP. Prototypes rather than simulations will be necessary to assess these risks with high confidence.

The per-component view of these risks (latency overheads in the Ledger, Network, and Mempool components) is treated in the respective sections below.

6.3 Network

The Network layer implements the mini-protocols that enable the Consensus layer to satisfy its diffusion requirements (**REQ-DiffuseLeiosBlocks**, **REQ-DiffuseLeiosVotes**) and prioritization requirements (**REQ-PrioritizePraosOverLeios**, **REQ-PrioritizeFreshOverStaleLeios**) defined in the Resource management section. While Consensus drives the scheduling logic for when to diffuse blocks and votes, Network provides the protocol mechanisms to actually transmit them over the peer-to-peer network.

> [!WARNING]
>
> TODO: Write out the actual risk and requirements from above, and what it means on the network layer

- **RSK-LeiosNetworkingOverheadLatency**: Same as RSK-LeiosLedgerOverheadLatency, but for the Diffusion Layer components handling frequent 15000% bursts in a caught-up node.

6.3.1 Message latencies

Due to extra volume that Leios imposes on the protocol, it is imperative that the underlying TCP bearer is managed such that arbitrary amount of data does not accrue in the host OS kernel buffers. Such a design is necessary to mitigate head-of-line blocking effects which would adversely affect network latencies, and in particular apparent ranking (Praos) block propagation times. For example, Linux and macOS support TCP socket option TCP_NOTSENT_LOWAT which allows to limit the volume of data written to the socket and that which cannot be yet sent out. This is very useful when we know what we want to diffuse, but also change our mind in terms of message ordering at the last moment, and do in the a way which doesnt artificially limit our throughput. Crucially, this option allow us to satisfy ranking block delivery timeliness guarantees which simultaneously gives us the means to manage the memory footprint of a running node. Sadly, this option is not universally supported and alternatives themselves are not portable, while manual transmission pacing is onerous so we postpone the investigation of the latter.

> [!WARNING]
>
> TODO: investigate possibility to use TCP_NOTSENT_LOWAT on cardano network despite its non-portability.

Benchmarks results showed that incremental decoding of a full Praos blocks can improve decoding time by several milliseconds (which is a minor improvement). However, more substantial gains were observed for blocks of size of several MBs, where time saved can reach several hundreds milliseconds (ref. Intersect/ouroboros-network#5367).

6.3.2 Traffic prioritization

The existing multiplexer is intentionally fair amongst the different mini-protocols. In the current CIP, the Praos traffic and Leios traffic are carried by different mini-protocols. Therefore, introducing a simple bias in the multiplexer (**NEW-LeiosPraosMuxBias**) to prefer sending messages from Praos mini protocols over messages from Leios mini protocols would directly enable the Consensus layer to satisfy **REQ-PrioritizePraosOverLeios** and mitigate **RSK-LeiosPraosContentionNetworkBandwidth**. This multiplexer bias is the primary candidate mechanism to ensure that Praos traffic and computation are prioritized over Leios, so that the diffusion and adoption of any RB is only negligibly slower.

> [!WARNING]

> TODO: How much prioritization is actually needed? The existing fair multiplexer may already be sufficient to keep Praos > Leios in practice, in which case **NEW-LeiosPraosMuxBias** can be skipped or weakened. Prototypes should measure this before committing to a strong (or full) Praos-over-Leios bias.

It is not yet clear how best to mitigate **RSK-LeiosLeiosContentionNetworkBandwidth** or, more generally, how to enable the Consensus layer to satisfy **REQ-PrioritizeFreshOverStaleLeios** (aka freshest first delivery) in the Network Layer. One notable option is to rotate the two proposed Leios mini-protocols into a less natural pair: one would send all requests and only requests and the other would send all replies and only replies. In that way, the server canwhen it has received multiple outstanding requests, which seems likelying during ATK-LeiosProtocolBurstreply to requests in a different order than the client sent them, which is inevitable since the client will commonly request an EB as soon its offered, which means the client will request maximally fresh EBs after having requesting less fresh EBs. If the client were to avoid sending any request that requires a massive atomic reply (eg a MsgLeiosBlockTxsRequest for 10 megabytes), then the server can prioritize effectively even without needing to implement any kind of preemption mechanism. This option can be formulated in the existing mini protocol infrastructure, but another option would be to instead enrich the mini-protocol infrastructure to somehow directly allow for server-side reordering. Whether any of this is needed requires further investigation through prototypes (EXP-LeiosDiffusionOnly).

6.3.3 High-throughput transaction submission

Leios raises the protocols target consensus data rate well above Praos. Since the available transaction volume always exceeds what consensus can include, the transaction submission layer between mempools must sustain throughput that exceeds the target consensus data rate otherwise the high-throughput mempool will starve and EBs will not fill.

- **REQ-LeiosTxSubmissionThroughput** The transaction submission protocol must sustain a data rate exceeding the target Leios consensus data rate across realistic peer valencies.

Current cardano-node (10.7, at the time of this writing) is by default using the legacy transaction submission protocol which fetches all transactions from every peer that offers them, even if some of those transactions are repeated. This scheme is found to be effective and robust in the current protocol implementation, but it necessarily leads to higher bandwidth consumption. A new protocol version v2 is being rolled out and tested, which is expected to bring cpu, memory and bandwidth use down, which in turn frees those resources for other tasks, and Leios in particular. Of the v2 variants, the v2 undecision variant has shown the highest sustained data rates across realistic peer valencies and is therefore the candidate that best fits **REQ-LeiosTxSubmissionThroughput**.

6.3.4 New mini-protocols

The node must include new mini-protocols (**NEW-LeiosMiniProtocols**) to diffuse EB announcements, EBs themselves, EBs transactions, and votes for EBs. These protocols enable the Consensus layer to satisfy **REQ-DiffuseLeiosBlocks** and **REQ-DiffuseLeiosVotes**. The Leios mini-protocols will require new fetch decision logic (**NEW-LeiosFetchDecisionLogic**), since the node should not necessarily simply download every such object from every peer that offers it. Such fetch decision logic is also required for TxSubmission and for Peras votes; the Leios logic will likely be similar but not equivalent.

> [!WARNING]
>
> TODO: anything generic to add on top of what is in the CIP? if not -> no point having this section and instead describe the network protocols (and how they make sense for our node) in the respective consensus sections

6.4 Consensus

CIP-0164 implies functional requirements for the node to issue EBs alongside RBs, vote for EBs according to the rules from the CIP, include certificates when enough votes are seen, diffuse EBs and votes through the network layer, and retain EB closures indefinitely when certified. The Consensus layer is responsible for driving these operations and coordinating with the Network layer (which implements the actual mini-protocols) to ensure proper diffusion.

6.4.1 High-throughput mempool

The Mempool already accepts transactions, diffuses them to peers, and pre-computes their validation for block production. What changes under Leios is that it must do all three at once, at a size where the way they are currently serialised stops working. Praos loads hide that; Leios does not, because it requires a much bigger Mempool (**UPD-LeiosBiggerMempool**). It must hold enough valid transactions for a block producer to fill a full EB alongside a full RB, at least twice an EBs capacity, so that a stake pool issuing a `CertRB` for a full EB can still fill a fresh EB alongside it (a `TxRB` carries fewer transactions than the EB certified by a `CertRB`). Stake pools are in any case indirectly incentivised to maximise EB size, just as with `TxRB`s, so that more fees enter the epochs reward calculation.

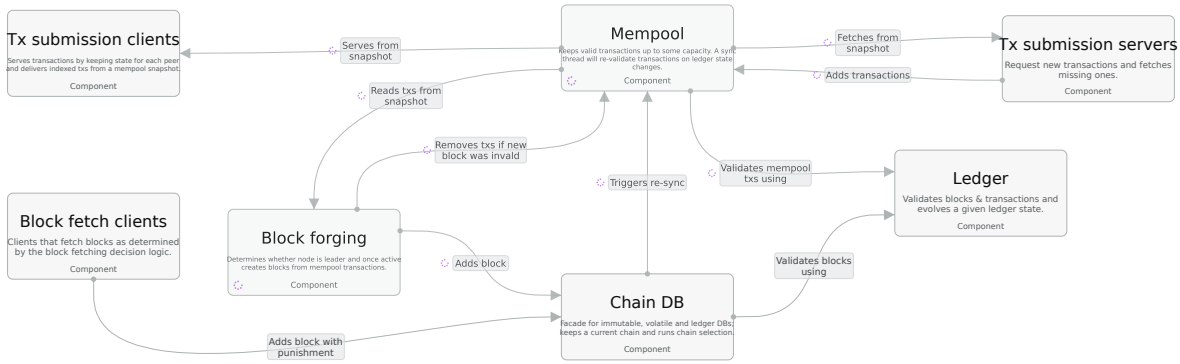
That very size is also what creates the challenge: the Mempool now holds roughly two endorser blocks worth of transactions instead of two Praos blocks, and it is fed at up to ~1000 tx/s. Re-processing that many transactions whenever the chain moves, while serialising every Mempool operation behind a single lock, blocks transaction diffusion and block production for as long as the re-processing takes (**RSK-LeiosMempoolOverheadLatency**).

Ad-hoc runs on a local twelve-node devnet, not a published benchmark, suggest that this risk does not degrade gracefully. Comparing two runs that differ only in effective Mempool capacity, roughly two and a half endorser blocks worth against nearly eight, the deeper Mempool *inverts*

the relationship between offered load and confirmed throughput: raising ingestion by a factor of two and a half lowered on-chain throughput by a third, whereas the shallower Mempool rose monotonically over the same load steps. Notably, the median cost of preparing a block was indistinguishable between the two runs; the entire difference sat in the tail. A design that bounds this work on average, rather than in the worst case, would therefore look correct in a benchmark and still lose leader slots in practice.

Two points of interpretation matter for the design. First, nothing in the protocol makes a large Mempool expensive; it is expensive because the implementation still performs work proportional to Mempool contents while a block is being produced, which converts depth into forging latency. Holding more transactions is precisely what fills endorser blocks, and the shallower configuration paid for its stability with a measurable throughput loss at low load. The objective is therefore not a small Mempool but a cheap large one, with a capacity chosen for memory and fairness rather than one that silently keeps forging affordable. Second, if the work on the critical path cannot be made sufficiently sublinear, an explicit governor is preferable to a capacity limit that happens to serve as one.

The solution is a threefold design: removing the lock contention so that accepting, diffusing and forging proceed concurrently (the *double-buffered mempool*), exploiting transaction validity ranges to make advancing the slot cheap, and keeping ready-to-forge views so that block production never re-applies the whole mempool on the critical path. The data structures and function names below track the `ouroboros-consensus` implementation and may evolve.



6.4.1.1 Double-buffering A transaction is fully validated only once, when it is first admitted (`applyTx`), and thereafter only re-applied against a changed base (`reapplyTx`), which is far cheaper because it reuses that validation and only re-checks that the transaction still applies (see Transaction validation levels).

This asymmetry is currently used by the mempool, and the double-buffering design leans on it further: a change of base forces `O(|txs|)` re-applications, but each one is cheap, and in particular cheaper than the full validation that concurrent additions are paying at the same time.

Cheaper does not mean free of ledger access, however. A new selection may have consumed UTxO that a transaction in the Mempool also consumes, so that transaction no longer applies, and the

only way to detect this is against the new selections UTxO map, which under UTxO-HD may require reading from disk via its `Forker`. Re-application is therefore cheap in validation work but not necessarily in I/O, and a design that assumes an entirely in-memory rebase would be wrong. Since a sync must touch every transaction, this is also where its cost is least predictable, which is a further argument for keeping it off the path that readers and block production depend on.

Model the Mempool as a single shared state, read by many and mutated by a few operations. The state is a set of transactions applied on top of a ledger state:

```
data MempoolState = MempoolState
  { base  :: LedgerState  -- the selection's ledger state the mempool sits on top of
  , txs   :: [Tx]         -- resolved, validated, applicable transactions, in order
  , after :: LedgerState  -- base with all of txs applied; what a new tx is validated agains
  }

data MempoolAction
  = AddTx  Tx             -- validate and apply a new tx (~1000/s)
  | Sync LedgerState      -- re-apply all txs onto a new base (a few per 20s)
  | Remove [TxId]         -- force-drop txs to recover from forging a rejected block (rare)
```

Note that a new transaction is not validated against `base` but against `after`, the state that results from applying every transaction already in the Mempool, in order, on top of `base`. That is what allows a chain of dependent transactions to be accepted, and it is why the two fields have to be distinguished.

Reading must never block: diffusion serves `txs` to peers, and block production draws from them to fill blocks. Reads also outnumber mutations, though not because transactions arrive slowly. Each admitted transaction is offered to every downstream peer, and a node may have a few hundred of those, so one `AddTx` gives rise to many reads; block production adds more. Adds and reads are coupled, but with a fan-out between them.

`AddTx` validates and applies a transaction against `after` (`applyTx`), appends it, and advances `after`. `Sync` takes a new `base` from chain selection and re-applies (`reapplyTx`) every transaction onto it, rebuilding `after` as it goes. The invariant both preserve is that a transaction is in `txs` exactly when it is valid and applicable in that order on top of `base`, which is also the Mempools core question: which transactions to accept, diffuse, and allow into a block.

`Sync` is the hard case: re-applying every transaction is `O(|txs|)`, and under Leios `|txs|` is large. Serialising it under one lock is exactly the bottleneck: while a sync runs, nothing can be accepted, diffused or forged.

The design that removes this contention is a **double-buffered mempool**, the same trick a graphics pipeline uses to keep a display smooth: readers are always served from the buffer **on screen**, the expensive work happens on a second buffer **off screen**, and the two are exchanged by a single quick flip.

- **On screen** is the committed state, the single authoritative buffer and the only commit point. Readers (block production, diffusion) are served from it without blocking each other; it is held for writing only briefly, to commit an `AddTx` or to flip in a completed `Sync`. Because there is exactly one on-screen buffer, size and capacity accounting has one source of truth and cannot diverge.
- **Off screen**, `Sync` re-applies every transaction against the new `base`, the `O(|txs|)` work, without holding the on-screen buffer or blocking anyone.

Both mutations commit through one operation, which reconciles the state they prepared off screen against whatever is on screen at that moment:

- `AddTx` validates its transaction off screen (the expensive, adversary-facing work), then commits it on screen by appending. Adds are serialised, so each commit is cheap.
- `Sync` re-applies all `txs` off screen, then runs a **converging loop**: it reads the transactions that landed on screen meanwhile (the delta) and re-applies only those on top, repeating until the delta drops below a target size or a maximum number of iterations is reached. The delta tends to shrink each round, because ingestion pays full validation whereas `Sync` only re-applies, which is cheaper per tx, and ingestion is serialised; but a steady stream of adds could keep it from ever reaching zero, so the iteration cap guarantees the loop always terminates. Only then does it take the on-screen buffer, re-apply that small residual under the lock, and flip to the new `base`. Since the residual never exceeds the target size (or the cap), the screen is held for a bounded time, independent of `|txs|`.

The result is still correct: the on-screen state is identical to re-applying all transactions against the new `base` in one pass, because re-apply is an ordered fold and splitting then resuming it yields the same result. And committing through the blocking flip, rather than an optimistic compare-and-swap, keeps a sync always making progress, never starved by a stream of concurrent adds.

> [!WARNING]
>
> TODO: Shouldnt we be more confident that our own constructed block is valid?

One mutation breaks the purely additive picture the converging loop relies on. `Remove` force-drops a given set of transactions *even though they are still valid*. It exists only for a rare inconsistency: the node forged a block the ledger then rejected, so those transactions must leave the Mempool or they would be forged into the same rejected block again. A `Sync` cannot stand in for it, since re-application would find the transactions valid (that is how they entered a block in the first place) and, the block having not been adopted, there is no new `base` to sync. Because `Remove` deletes rather than appends, an in-flight `Sync` that snapshotted before it would resurrect the deleted transactions when it flips. The double-buffer keeps the loop additive by **preempting**: a `Remove` commits on screen and restarts any in-flight `Sync` from a fresh, post-removal snapshot. This is cheap precisely because `Remove` is a rare recovery action, never the normal way forged transactions leave the Mempool (that is an ordinary `Sync` once the node adopts its own block); were removals to become frequent, replaying an additions-and-removals log during the sync would be preferable to restarting it.

6.4.1.2 Cheap ticking

> [!WARNING]
>
> TODO: Not yet implemented and incomplete design. How will this exactly result in less checking in the forge loop? Shouldnt we incorporate knowledge of the leader schedule?

A transaction is applicable only over a **validity range** of slots: its own validity interval (`invalidBefore`/`invalidHereafter`), together with the era and protocol parameters that govern how it is validated. So advancing the slot, not only changing the `base`, can invalidate a transaction.

Two facts keep this cheap to track. The era and protocol parameters change only at **epoch boundaries**, so within an epoch the validation rules are fixed. And the mempool commits to an `interval` of its own, a window reaching to the next expected block or the epoch boundary, admitting only transactions valid across the whole of it:

```
data MempoolState = MempoolState
  { base     :: LedgerState
  , interval :: (SlotNo, SlotNo)  -- slot range the whole mempool is valid over
  , txs      :: [Tx]
  }
```

A third operation, `Tick`, advances the current slot as a cheap **intersection**: as long as the new slot stays within `interval`, nothing observable to the transactions changes and nothing is re-applied. Leaving the `interval`, at the latest at an epoch boundary, forces a re-apply (and a re-validate on a new era). Since `interval` is roughly a 20-slot window, from the current tip or epoch start to the next expected block or epoch end, maintaining it lets the node advance many slots between the bigger, base-changing rebases without ever touching `txs`. The trade-off is that a transaction whose own validity range is narrower than `interval` cannot be admitted, which is acceptable for the performance win.

6.4.1.3 Ready-to-forge views

> [!WARNING]
>
> TODO: Not yet implemented and likely incomplete design

Removing the lock contention lets forging run concurrently with ingestion and diffusion, but it does not by itself make forging fast: the block producer still needs the transactions applicable at the forging slot, and computing that naively means re-applying the mempool at that moment, the `O(|txs|)` cost we must keep off the critical path where only a fraction of a slot is available.

For forging, a block producer works against not one `base` but two. Under Leios in Dijkstra it issues an EB alongside each RB, and that RB either certifies the EB announced by the preceding RB (a `CertRB`) or carries transactions directly (a `TxRB`). Those two outcomes are two different bases the mempools transactions would be applied on:

- one where the preceding endorsement is **not** being certified, so all mempool transactions are available to forge and announce;
- one where it **is** being certified, so the transactions of that now-settled EB are already in the ledger and must be excluded.

Which base applies is only decided at forge time, once it is known whether enough votes for the preceding EB have arrived. The Mempool therefore keeps **two ready-to-forge views**, one per base, pre-computed so that neither outcome pays a re-application on the critical path. At forge time the block producer just picks the matching view, selects transactions, and performs the cheap slot-dependent checks (see cheap ticking).

For that to hold, forging must stay a cheap *read* of a view and never turn into a fresh computation. Two mechanisms keep it so, each with a concrete form in the `cardano-node`:

- an **optimistic view** keeps forging off the rebase path: the block producer reads the matching ready-to-forge view as it stands, just as any other reader is served from the on-screen buffer, rather than first synchronising it to the very latest base. A tip that arrived moments earlier is picked up on the next block-production opportunity.
- a **capped forging snapshot** bounds whatever reconciliation is still unavoidable: advancing the chosen view to the forging slot is free while it stays within the `interval` (cheap ticking), and any residual re-application against the base (today `getSnapshotFor` can trigger a full one) is time- and size-bounded, so producing the snapshot cannot stall behind a large re-apply.

Of the two, the cap is the harder guarantee, because it is what bounds the case the optimistic view does not cover. Reconciliation, not the selection of transactions, dominates the cost of producing a block, and a view that is free when it is current but unbounded when it is not still puts seconds into a leader slot. Bounding the expected cost is therefore not sufficient; the bound has to hold precisely when the view must be brought forward, which at Leios Mempool depths is a common enough case to dominate the distribution.

Missing it costs more under Leios than under Praos, where a slow forge costs at most the slot. An endorsement can only be certified by the block succeeding the one that announced it, so the votes it needs must be produced and diffused within a single inter-block interval. A slow forge spends part of that interval before anyone else can begin: the announcement, and the EB data a voter needs in order to validate what it is voting on, are released late by however long the producer took to prepare them. Every voters window is shortened by that delay, not just the producers own, so votes arrive later and some arrive too late to count, taking with them the endorsement they would have certified. Forging latency therefore does not merely delay a block, it propagates into the voting round and can amplify into certification failures across the network, which argues for keeping this work off the critical path rather than merely making it faster.

[!WARNING]

TODO: This work grows with transaction *size* as well as with their number, so a bound expressed purely in transaction count would not constrain it. The bound presumably wants to be in terms of the ledger work implied by a view, though what the right measure is remains open.

[!WARNING]

FIXME: Write about ingesting txs into the mempool from endorsements (to the end of it). This should actively work against front-running and mempool fragmentation (the latter may be induced by network delay / partitions, while the latter is with zero delay and just reapplying txs from the cache)

6.4.2 Block production

The existing block production thread must be updated to generate an EB at the same time it generates an RB (**UPD-LeiosAwareBlockProductionThread**). In particular, the hash of the EB is a field in the RB header, and so the RB header can only be decided after the EB is decided, and that can only be after the RB payload is decided. Moreover, the RB payload is either a certificate or transactions, and that must also be decided by this thread, making it intertwined enough to justify doing it in a single thread.

- **REQ-IssueLeiosBlocks** The node must issue an EB alongside each RB it issues, unless that EB would be empty.

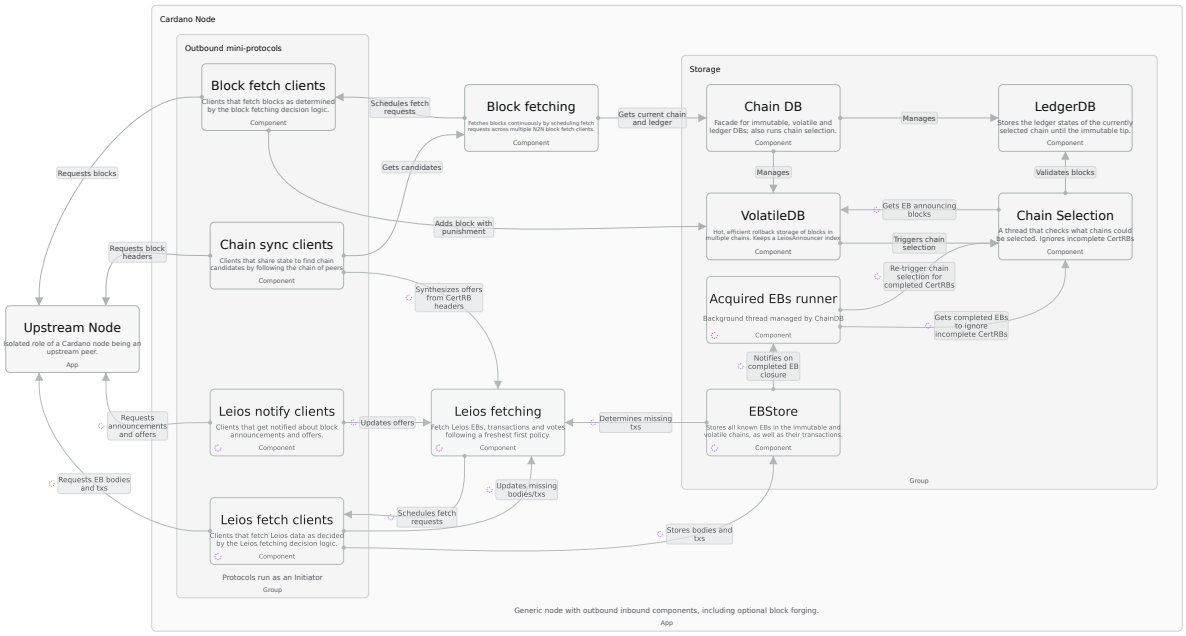
For the block production thread to determine which transactions from a mempool snapshot can go into an RB, which overflow into an EB and how many can fit into the EB, a new capacity measure analogous to the existing `blockCapacityTxMeasure` will be needed for EBs (**NEW-LeiosEbCapacityMeasure**). The EB capacity measure must incorporate the new EB-specific block limit protocol parameters from the ledger state, much like the existing Praos block capacity is also determined by protocol parameters.

Furthermore, the existing `forgeBlock` method and/or the `BlockForging` interface must be extended (**UPD-LeiosForgeBlock**) to optionally produce an EB for a given block type (`blk`). The transactions to include are prepared by the Mempool ahead of time, see ready-to-forge views.

[!WARNING]

FIXME: also write about including certificates in blocks when available, also link to certification and subsequent sections on chain selection/block validation

6.4.3 Endorser block diffusion



[!WARNING]

FIXME: what kind of validation while diffusing (only size and hash checks)

- **REQ-DiffuseLeiosBlocks** The node must acquire and diffuse EBs and their closures (via the Network layers new mini-protocols, see below).

6.4.3.1 The crucial attack vector Threat vector #22 forces this decision logic to aggressively issue hedge requests. The T22 attack is only mitigated if the network reliably diffuses the EB from the ~25% of stake that is honest, already has the EB, and might be geographically concentrated (for example, in Europe) to the rest of the honest stake spread around the globe in less than approximately $L_{\text{diff}} +$. With typical parameter values, thats $7 + 5 = 12$ seconds. Because of the unavoidably high RTTs across the global spans, the widespread convention of packet loss as a congestion signal, and the severe consequences of loss in the prevailing TCPs CUBIC/etc congestion avoidance algorithms, that 12 seconds does not include much slack for transporting payloads upwards of 12 MB over public Internet infrastructure while tolerating a large number (but $< 100\%$) of potentially adversarial peers. Moreover, Leios traffic is inherently bursty, so the common convention of resetting the congestion window during idle periods additionally reduces goodput. (BBR might outperform CUBIC/etc, but not by a whole order of magnitude, and BBR also carries its own risks.)

Because the deadline is tight and adversarial peers can appear to be strong honest peers before an abrupt betrayal, timeouts are insufficient. No timeout will both robustly prevent a *slow loris* attacker from wasting a problematic portion of the $L_{\text{diff}} +$ budget and also robustly avoid false alarms: over the public Internet, distant peers, even if honest, will exhibit unpredictable

changes in RTT and/or goodput performance. For similar reasons, any fetching policy that relies on *promptly* reacting to peer responsiveness is inapplicable.

The only robust mechanism is to leverage multiple peers simultaneously, akin to *hedge requests*, popularized by *The Tail at Scale* (CACM 2013), but maybe even issuing the hedge requests without waiting for a (robust) signal that theyre actually necessary. Timeouts alone are insufficient but still worthwhile, only for the sake of reclaiming resources from an entirely unresponsive peer on a reasonable time scalenot as a means to reactively minimize the latency of an inflight payload.

In particular, the fetch decision logic will leverage the fact that each of its stake-sampled peers has a 25% probability of having T22s EB no later than the start of the $L_{\text{diff}}+$ duration. Each Cardano node maintains 5 stake-sampled upstream peers (a.k.a. big ledger peers) (TODO confirm). These peers are subject to churn, which means the peers that have survived churn are not *independent* samples of the pools. However, at least one of them is independent: the one sampled at the latest churn. A pools block-producing node (which is also where Leios votes originate) should be running behind *at least* 3 relays (TODO confirm). So, in total, a block-producing node is effectively one hop away from 15 stake-sampled peers and at least 3 of those samples are independent. Thus, each honest pool endangered by a T22 attack has *at least* a 1 - (1 - 25%)^3 = 57.8% probability of being just one hop away from the certified EB at the start of the $L_{\text{diff}}+$ duration. (TODO the Fetch logic can and should give special treatment to a block-producers own relays: theyll have a good connection and theyre *trustworthy*. Also, the relay can start serving downstream peers even before its block-producer finishes downloading. Together, these justify not counting it as a hop.)

- Because the maximum pool size is 0.2% stake (i.e. `nOpt` is 500), all of those requests will be spread across more than a hundred pools (i.e. many hundreds of relays) rather than bottlenecking.
- After that one hop, the probability of being merely one more hop away compounds; that snowball effect lets the EB diffuse within a few hops, which makes it plausible for the EB to reliably diffuse before $L_{\text{diff}}+$.

For the sake of the LeiosFetch decision logic, the key observation is that having connections to those upstream peers is only useful if the logic actually utilizes them. The LeiosFetch decision logic thus requests all desired data from every stake-sampled peer that offers it, with no effort at avoiding redundant downloads among them. This is the only way to robustly prevent a slow loris attacker from wasting some of the precious latency budget.

6.4.3.2 Redundant downloads

The fetch logics aggressive hedge requests incur one major downside: each node will often download multiple copies of Leios data. The previous section motivates that redundancy in order to prevent T22 from disrupting the base Praos protocol, which is crucial. Still, it is undesirable to waste the downstream peers ingress resources and the upstream peers egress resources. Some ways to lessen the redundancy have been considered.

- However much of an EB has already arrived before a specific upstream peer offers that content wont be downloaded again. Redundant fetches will only be sent for what hasnt already arrived. Most of the time, honest upstream peers will not all simultaneously offer the EB, so the naturally gradual diffusion alone will prevent some of the possible redundancy.
- One refinement would be to send best-effort cancellations, at least to the stake-sampled peers. The cancellation cannot be assumed to succeed, due to the inherent race condition, but it may reach the server in time to trivialize some of its responses (e.g. merely send an empty reply). (TODO note that these cancellations would need to be able to skip the queue on both the sending node and the receiving node to have an effect on how the preceding requests are handled. For example, that might motivate sending the cancellations via a different mini protocol than the requests.)
- Another refinement could be to introduce a *small fixed* delay before issuing any hedge requests. Its not already clear whether this will waste too much of the latency budget, so it may be worth considering later. Key concern: nothing prevents the adversary from being the first to *offer* some desired data, which means the T22 attacker can always abuse this delay.
- The fetch logic will also be utilizing the nodes ~20 upstream peers that are sampled from PeerSharing rather than from the stake distribution. With such a high valency, these peers have a higher chance of being a geographic neighbor, which correlates with significantly better RTT and bandwidth. As such, these peers can transfer even a 12 MB payload very quickly despite in contrast to assumed-to-be-intercontinental stake-sampled peers a relatively tight bound on outstanding replies. The intended timeline is that stake-sampled peers ensure the EB is one or two hops away from every neighborhood and then the PeerSharing-sampled peers ensure the EB spreads rapidly within each neighborhood. As a result, its more likely that the data that arrived quickly via PeerSharing-sampled peers either wont need to be requested from a stake-sampled peer at all or could be quickly cancelled.

6.4.3.3 Utilizing PeerSharing-sampled peers

Like stake-sampled peers, the fetch logic utilizes every PeerSharing-sampled peer that offers the desired data. However, it does bound the outstanding replies to approximately 1 MB, so that even utilizing ~20 peers does not consume an infeasible amount of in-memory buffers and other ingress resources.

Due to that finite budget, the fetch logic must choose which requests to spend it on for each peer. The most useful scheme seems to be RarestFirst with arbitrary tiebreakers: choose one of the requests that is inflight with the fewest other peers. Thus, while stake-sampled peers requests are decided entirely independently, the PeerSharing-sampled peers exhibit a small amount of interaction: their budget is always saturated regardless of other peers state, but which particular requests are sent is dependent. (TODO mention that stake-sampled peers can request the closures txs in any order, e.g. randomized?)

6.4.3.4 Deciding among multiple offers

The above is a full description of how the fetch logic would utilize all of the nodes upstream peers to acquire a single EB. But the fetch logic must also handle multiple EBs. Unfortunately, Leios does allow for there to be so many EBs diffusing simultaneously that the fetch logic cannot simply repeat the above for all EBs separately. Thatd be a truly unbounded amount of outstanding replies per stake-sampled peer and numEbsOfferedŒ1 MB per PeerSharing-sampled peer.

Instead, there is a fixed budget of how many EBs can be inflight with a peer at once, for example three for a stake-sampled peer and five for a PeerSharing-sampled peer. The size of outstanding replies for each PeerSharing-sampled peer would thus be at most min(5, numEbsOffered)Œ1 MB at most ~100 MB among the ~20 PeerSharing-sampled peers. The total is similar for the stake-sampled peers: min(3, numEbsOffered)Œ12 MB is ~180 MB among the five of them. Due to that finite budget, the fetch logic must choose which EBs to spend it on for each peer. For **REQ-PrioritizeFreshOverStaleLeios**, to resist **ATK-LeiosProtocolBurst**, this prioritization is FreshestFirst with arbitrary tiebreakers: choose one of the EBs that is announced in the greatest slot among the EBs offered by this peer. If the peer offers a higher-priority EB while this budget is saturated, the fetch logic should immediately stop sending requests for the least-prioritized of its inflight EBs (i.e. drain its replies) to rapidly make room for the new most-prioritized offer (TODO it is not obvious whether sending cancellations to possibly accelerate this drain would be effective enough to be worth the complexity/extra messages).

TODO refine to FreshestLast among the most recent L slots and FreshestFirst among older slots in order to also address RSK-LeiosProtocolStorm

6.4.3.5 Tolerating equivocation even despite L_{hdr} violations

The Leios protocol mitigates equivocation by relying on the prompt diffusion of announcements within L_{hdr}. The following hold when that rapid diffusion is achieved.

- A node should only fetch the first announcement it sees from each *election* (i.e. a SlotNo-PoolId pair). The second and subsequent announcements it sees are proof of that elections equivocation and should otherwise be ignored. Thus, honest nodes only ever fetch one EB per election.
- The voting rules prevent the certification of any announcement from an election if that election was equivocated soon enough to risk having honest nodes disagree on which announcement was first. Thus, honest nodes only ever fetch different EBs for the same election when there can be no certificate for any of that elections announcements.

It is plausible that L_{hdr} will be violated infrequently and only for a few nodes at a time. However, L_{hdr} violations are definitively not impossible; no real-world networking infrastructure could perfectly mitigate that risk (for any feasible value of L_{hdr}). Ideally, the node can automatically recover, which requires some honest nodes to fetch a second EB for that election. But *never* a third EB, because even despite L_{hdr} violations, only a single EB could be certified for each election.

This section specifies the diffusion of announcements and the recovery path for when L_{hdr} violations cause an honest node to need to fetch a second EB for some election.

On-time announcements are the most urgent Leios messages to relay. Fortunately, an announcement is no greater than 1 kB, and so definitely fits in a single modern TCP packet. Thus, the only mechanism for undue delay would be buffer bloat arising from these messages being enqueued behind bigger messages. The global buffer bloat initiative has proceeded well, such that even public Internet infrastructure paths (which are not owned/controlled by Cardano node operators) tend to avoid excess buffering of TCP traffic. And the techniques described in the Message latencies subsection above prevent such buffers from arising within the node itself. Moreover, the nodes own multiplexer mitigates head-of-line blocking between different protocols, which motivates relaying announcements in a separate mini protocol for any large payloads: the LeiosNotify-LeiosFetch distinction (see the mini protocols specified below).

Without L_{hdr} violations, an honest node never has reason to relay more than two announcements per election: the first because its eligible and the second to prove equivocation (which affects the voting rules). Moreover, the honest node only has reason to offer to relay the EB body and EB closure of the first announcement it relayed. These invariants symmetrically allow the node to disconnect from an upstream peer sending junk/too many announcements and/or offers: a first announcement, possibly a second for equivocation proof, an EB body offer for the first announcement, and an EB closure offer for the first announcement. Nothing more needs to be sent for each election.

In the case of L_{hdr} failure, however, the node may need to offer the EB body and closure of an announcement different from the first announcement it relayed. One possibility would be to allow upstream peers to send a (potentially-)third announcement and also to subsequently offer its EB body and/or EB closure. However, the need for those additional messages cannot be validated before the recipient has validated a certificate for that third announcement. This suggests an alternative to a second LeiosNotify round of announcement and offers for the purpose of recovery: recover instead via ChainSync and BlockFetch, because in the Linear Leios design only BlockFetch delivers certificates.

In the current design, Leios announcements are Praos headers. Therefore, ChainSyncs MsgRollForward inherently carries announcements. Additionally, the current design includes a new bit in the Praos header allowing it to claim that its Praos body contains a Leios certificate for the predecessors announcement. Together, this information is sufficient to enable the recovery path for the occasional node that needs to offer and/or fetch a second EB for some election. (TODO If we change the design so that RbHeaders are no longer announcements, then the cert bit could be expanded to include the certified announcement, in which case all the necessary values are still in MsgRollForward.)

If the certified EB on a desirable Praos chain is different than the first announcement seen for that same election, then the cert-carrying Praos block (a.k.a. CertRB) will arrive before the certified EB. Once the node validates the certificate, it can then do the second round of fetching. Some of its peers may have originally offered the certified announcements EB. Any peer that instead relayed a wrong announcement for this election could also offer the certified

announcments EB simply by sending the two MsgRollForwards (which it will already do for the sake of ChainSync): the first announces the EB and the second claims to have selected the CertRB, which is impossible unless the peer also has both the EB body and EB closure. Hence the second MsgRollForwards set cert bit is effectively an offer of the first MsgRollForwards announced EB body and closure.

TODO bonus: this same mechanism also works for connections that arise *after* the announcement was relayed

TODO

- The first version of LeiosFetch server simply pulls serialized transactions from the LeiosEbStore, and only sends notifications to peers that are already expecting them when the noteworthy event happens. If notification requests and responses are decoupled in a separate mini protocol *or else* requests can be reordered (TODO or every other request supports a MsgOutOfOrderNotificationX loopback alternative?), then itll be trivial for the client to always maintain a significant buffer of outstanding notification requests.
- Thats only true if there is never *any budget whatsoever* on outstanding requests. Otherwise, merely relaying the announcement would cause the upstream peer to consume some of its budget even for EBs it cant serve (since we send requests without waiting for offers).

6.4.3.6 Bounding incoming messages As mentioned for offers in the previous section, the node must be able to detect junk/excessive messages from its peers (**NEW-LeiosIncomingMessageBounds**). Otherwise, there would be trivial Denial-of-Service attack vectors.

In the current design, upstream peers send the following messages for EB diffusion.

- An *announcement* from this peer causes disconnection when any of the following hold.
  - This peer has already sent this same announcement.
  - This peer has already sent two distinct announcements for this election, so this one would be a third.
    - Recall: a second, distinct announcement for an election is how a peer proves that elections equivocation; there is no separate equivocation-proof message, so an honest node accepts at most two distinct announcements per election.
  - The announcement has an invalid signature.
    - Because CIP-0164 reuses Praos headers as EB announcements, the signature involves an operational certificate (opcert) issue number (OCIN). When ChainSync validates a MsgRollForward, this OCIN is upper bounded based on the preceding header chain, since headers themselves update the OCINs. In LeiosNotify, announcements are not part of a chain, and so precise validation of the OCIN isnt necessarily possible; the recipient might not have yet seen the header that increments the OCIN. A sufficient compromise is to reject OCINs that are less than

the OCIN recorded by immutable tips ledger state (which all nodes must agree on), but allow *any* OCIN at least as great. As a result, opcert revocation for EB announcements is delayed until the incremented OCIN becomes immutable, after ~12 hr when Chain Growth is healthy and up to ~36 hr when it isnt. Thats tolerable: it just means a leaked key allows the attacker to equivocate (hence mostly nullify) the victims EB announcements until their revocation becomes immutable. Crucially, ChainSync doesnt change its behavior: it still enforces the OCIN bounds on the specific header chain, which is as precise and prompt as possible.

    * (TODO hmm.. what if the attacker sends a *fresh* announcement that uses the revoked OCIN *just before* it becomes immutable? That might cause some honest nodes to disconnect from other honest nodes. An attacker pool could arrange for this opportunity once per header they get onto the chain, since they could increment their OCIN with every header. Perhaps the honest node accepts the revoked OCIN for, say, 1 extra minute, permitting its upstream peers imm tips to catch up?)

  - The announcement has an invalid election proof (i.e. VRF proof).
  - The contained Praos header doesnt actually announce an EB.
  - The announcements EB size is too great.
  - The announcements election is more than 10 minutes old; honest servers will skip relaying announcements older than 5 minutes and the extra 5 minutes accommodates transmission time and clock skew.
  - The announcements slot is more than clock skew (eg 2 seconds) from the future. If less than that, just pause this peers LeiosNotify client until the local wall clock reaches the onset of the announcements slot.

- An *offer* from this peer causes disconnection when any of the following hold.

  - The same content has already been offered by this peer.
  - The offered EB has not already been announced by this peer.
  - The offered EB has been announced by this peer but its (youngest) announcement is more than 10 minutes older than the local immutable tip; honest servers wont offer anything older than their immutable tip, and 10 minutes accommodates potential disagreements on whats immutableunless the upstream is a syncing node, which also justifies a disconnect. (A syncing honest node wont request notifications until its caught-up; see Catching up below.)

- An *EB body* from this peer causes disconnection when any of the following hold.

  - The node didnt send a request for (this copy of) it from this peer. For this revision, the corresponding request must in fact be the oldest body-or-closure request the node sent to this peer. (TODO a later revision might allow the peer to send replies to out-of-order higher priority requests first, when possible.)
  - Its contents do not match the hash and size listed in the announcement.

- An *EB closure portion* from this peer causes disconnection when any of the following hold.

  - The node didnt send a request for (this copy of) it from this peer. For this revision, the corresponding request must in fact be the oldest body-or-closure request the node has sent to this peer. (TODO a later revision might allow the peer to send replies to out-of-order higher priority requests first, when possible.)
  - Its contents do not match the full hashes and sizes listed in the EB body for the transactions in the corresponding request.

In the current design, downstream peers send the following messages for EB diffusion.

- A *notification request* causes disconnection when any of the following hold.
  - SN + 300 RN, where RN is the number of notification requests the node has received from this peer and SN is the number of notifications (announcements/offers) the node has sent this peer.
  - (TODO 300 is a magic number; the necessary value depends on the burst size, which needs additional assessment)
- An *EB body request* causes disconnection when any of the following hold.
  - This peer has sent body-or-closure requests whose cumulative own size (as opposed to the size of the corresponding replies) is obviously greater than it would ever need to be (2 MB buffer?) before this node has processed even one of them.
  - The node doesnt currently have that EB body in our Leios storage.
  - (Rate-limit is handled outside the node, e.g. `fail2ban`.)
- An *EB closure portion request* causes disconnection when any of the following hold.
  - This peer has sent body-or-closure requests whose cumulative own size (as opposed to the size of the corresponding replies) is obviously greater than it would ever need to be (2 MB buffer?) before this node has processed even one of them.
  - The node doesnt currently have that EB closure portion in our Leios storage.
  - (Rate-limit is handled outside the node, e.g. `fail2ban`.
- TODO how would cancellations be handled? Especially since they could arrive after we sent our reply :grimace:

6.4.3.7 Bounding in-memory state All of the new Leios state must require only bounded memory usage (**NEW-LeiosBoundedMemory**). The previous section bounds the number of incoming messages in terms of the number of upstream/downstream peers and (multiplicatively) the number of sufficiently young elections in the Praos leader schedule (which is reused as the Leios leader schedule), and the nodes memory usage must also be bounded.

The above requirements for detecting junk messages from upstream peers force the node to maintain state that summarizes which announcements/equivocation proofs/offers each upstream peer has already sent. The similar requirements for messages from downstream peers seem to only require much simpler state (merely a counter and a buffer size limit), which would be appealing because there are so many more downstream peers than upstream peers. Surprisingly,

because of symmetry, the node must unfortunately keep some similar state for downstream peers so that it can avoid sending messages that they would consider junk. For example, if a downstream peer connects after some announcement becomes too old to relay but before the node acquires the EB body, the node must not offer that body to that peer, since the node never sent the corresponding announcement to that peer. (It may be tempting to suggest that the node could send the announcement along with the offer in this case, but the announcement might be too old for the peer to accept it and whether the announcement has already been sent is the very state in question.)

Because the required state is not obvious, this subsection explicitly lists which in-memory state the above design requires. It also makes it obvious that this state can be trimmed as the immutable tip advances, by explicitly limiting itself to *volatile* elections, which are the elections that are not older than the current immutable tip.

- For EB diffusion, for each downstream peer, the node must track:
  - The difference between the number of notifications sent (SN) and number of notifications requested (RN). (If its too great, the node disconnects, so this isnt unbounded.)
  - Which volatile announcementsi.e. announcements for a volatile electionthe node has relayed to this peer. This varies per peer for two reasons: peers connect at different times and even a connected peer might not have had sent an unconsumed notification request when this node relayed the announcement. In particular, if the node was unable to send its first-seen announcement to a downstream peer, then it should also not send an equivocating announcement to that peer, since the peer would incorrectly interpret it as the nodes first-seen announcement for that election.
- For EB diffusion, for each upstream peer, the node must track:
  - Which requests the node has sent to this peer.
  - Which announcement this peer first sent for each volatile election.
  - Which volatile equivocation proofsi.e. proofs of equivocation of a volatile electionthis peer has sent.
  - Which volatile EBsi.e. EBs announced by at least one volatile announcementbodies and/or closures this peer has offered.

The node does not need to track which equivocation proofs or offers it has relayed to each peer, because the node only sends notifications when it first acquires the equivocation proof/body/closure, which fundamentally only happens once. The node does, however, need to track which equivocation proofs/bodies/closures have not yet arrived centrally for that reasonand more state as well for other behaviors specified above.

- For EB diffusion, the node centrally tracks:
  - The first announcement it saw for each volatile election.
  - The second announcement (a.k.a. equivocation proof), if any, it saw for each volatile election.

- Which volatile-announced EB bodies it has already acquired, so that it does not re-request bodies it already has.
- Which portions of volatile-announced EB closures it still needs to request, so that it does not re-request portions it already has.
- Which volatile-announced EB closures it has already acquired (see Chain selection below).
- Which RBs in the VolDB are CertRBs (see Chain selection below).
- Which EB, if any, each RB in the VolDB announces (see Chain selection below).
- Which announcement, if any, has been certified for each volatile election (see Block validation below), since that might cause this node to fetch a second EB (the certified announcements) for an election. (There cannot be more than one certified announcement per election unless the adversary controls >50% of the committee, which is already a disaster.)

All of this state is obviously bounded (e.g. the difference of SN and RN) or else ultimately traceable to the set of volatile elections. With some care (e.g. race conditions, accounting for transport time/clock skew, etc), that makes it trivial to bound this state as time passes: whenever the immutable tip advances, drop state thats only relevant to elections older (or, e.g., at least 10 minutes older) than the new immutable tip.

The number of volatile elections is stochastically bounded, so the above discharges the requirement. In practice, the number of such elections depends on the protocol parameters and the applicable risk tolerance. For Cardano mainnet, a sufficient quantity is 10,000 electionsthat incurs an error probability below 10^{30} even assuming a maximally old immutable tip (36 hr), a grinding adversary, a perfectly uniform infinite stake distribution, and multiple decades worth of slots.

TODO what would happen if theres 10,001 elections?

TODO other non-diffusion components, such as voting, should also contribute to this list/have their own

6.4.4 Endorser block storage

Unlike votes, a node should retain the closures of older EBs (**NEW-LeiosEbStore**), because Praos allows for occasional deep forks, the most extreme of which could require the closure of an EB that was announced by the youngest block in the Praos Common Prefix. On Cardano mainnet, that RB is usually 12 hours old, but could be up to 36 hours old before CIP-0135 Disaster Recovery Plan is triggered. Thus, EB closures are not only large but also have a prohibitively long lifetime even when theyre ultimately not immortalized by the historical chain.

- **REQ-StoreLeiosBlocks** The node must retain all EBs and their closures up to the immutable tip, independent whether certified or not.
- **REQ-ArchiveLeiosBlocks** The node must retain each EBs and their closures indefinitely when the immutable Praos chain certifies them.

This component therefore stores EBs on disk just as the ChainDB already does for RBs. The volatile and immutable dichotomy can even be managed the same way it is for RBs.

> [!WARNING]
>
> TODO: discuss sizing and access patterns - up to 11k block opportunities within 12h? -> see nfrisbys eb storage document TODO: separate volatile/immutable stores? why? - having an immutable EBs table or database should bound access times on EBs and closures of the volatile part (which should be log n, thus bounding n is desirable) TODO: interaction with mempool (here or above)

The first version of LeiosEbStore can just be two bog standard key-value stores, one for immutable and one for volatile. A second version maybe instead integrates certified EBs into the existing ImmDB? That integration seems like a good fit. It has other benefits (eg saves a disk roundtrip and exhibits linear disk reads for driver prefetching/etc), but those seem unimportant so far.

What justifies fetching an EB. An election or a certification does; a peers offer does not. An offer says only which peers can serve what, so nothing is listed to fetch on the strength of a claim the node has not verified, and the set of bodies it will pursue is bounded by elections rather than by what peers assert. Offers are correspondingly constrained: a peer may offer only the parts of an EB it has itself announced to this node, at most once per part, and never for an EB older than the nodes immutable tip.

- **REQ-FetchJustifiedByElection** The node must fetch an EB because an election or a certification calls for it, never because a peer offered it.

A corollary worth stating, because the obvious assumption is wrong: the ChainDB is *not* a durable record of announcements. Announcements diffuse independently of ChainSync, precisely so they can outrun it, so a header on a fork the node never selects, a header equivocating an election, or a header announcing an EB the node never fetches need not reach block storage at all.

A wrong declared size is not an attack to punish. If an announcement declares a size the EB does not have, the node simply never counts that closure as complete so it neither relays the closure nor votes for the announcement. Nobody is disconnected over it: the issuer is not a peer, and the peer that relayed the announcement may be perfectly honest.

An equivocated election is resolved by certification, not by guessing. Two announcements for one election leave the node unable to say which EB the chain will settle on. Rather than choose, it follows the certificate: a verified certificate names an EB, and that is what makes the node pursue that body including the case where an equivocation had previously pointed it at the other one.

6.4.5 Transaction cache

> [!WARNING]
>
> FIXME: Moved here, make sure it has good context. Keep it here before voting as we need it to determine (efficiently) whether to vote or not (an EB closure applies or not)

A new storage component (**NEW-LeiosTxCache**) will store all transactions received when diffusing EBs as well as all transactions that successfully enter the Mempool, up to some conservative age (eg one hour). The fundamental reason that EBs refer to transactions by hash instead of including them directly is that, for honest EBs, the node will likely have already received most of the referenced transactions when they recently diffused amongst the Mempools. Thats not guaranteed, though, so the node must be able to fetch whichever transactions are missing, but in the absence of an attack that ought to be minimal.

The Mempool is the natural inspiration for this optimization, but its inappropriate as the actual cache for two reasons: it has a relatively small, multidimensional capacity and its eviction policy is determined by the distinct needs of block production. This new component instead has a greater, unidimensional capacity and a simple Least Recently Used eviction policy. Simple index maintained as a pair of priority queues (index and age) in manually managed fixed size bytearrays, backed by a double-buffered mmapped file for the transactions serializations. Those implementation choices prevent the sheer number of transactions from increasing GC pressure (adversarial load might lead to a ballpark number of 131000 transactions per hour), and persistences only benefit here would be to slightly increase parallelism/simplify synchronization, since persistence would let readers release the lock before finishing their search.

Note: if all possibly-relevant EBs needed to fit in the LeiosTxCache, its worst case size would approach 500 million transactions. Even the index would be tens of gigabytes. This is excessive, since almost all honest traffic will be younger than an hourassuming FFD is actually enforced.

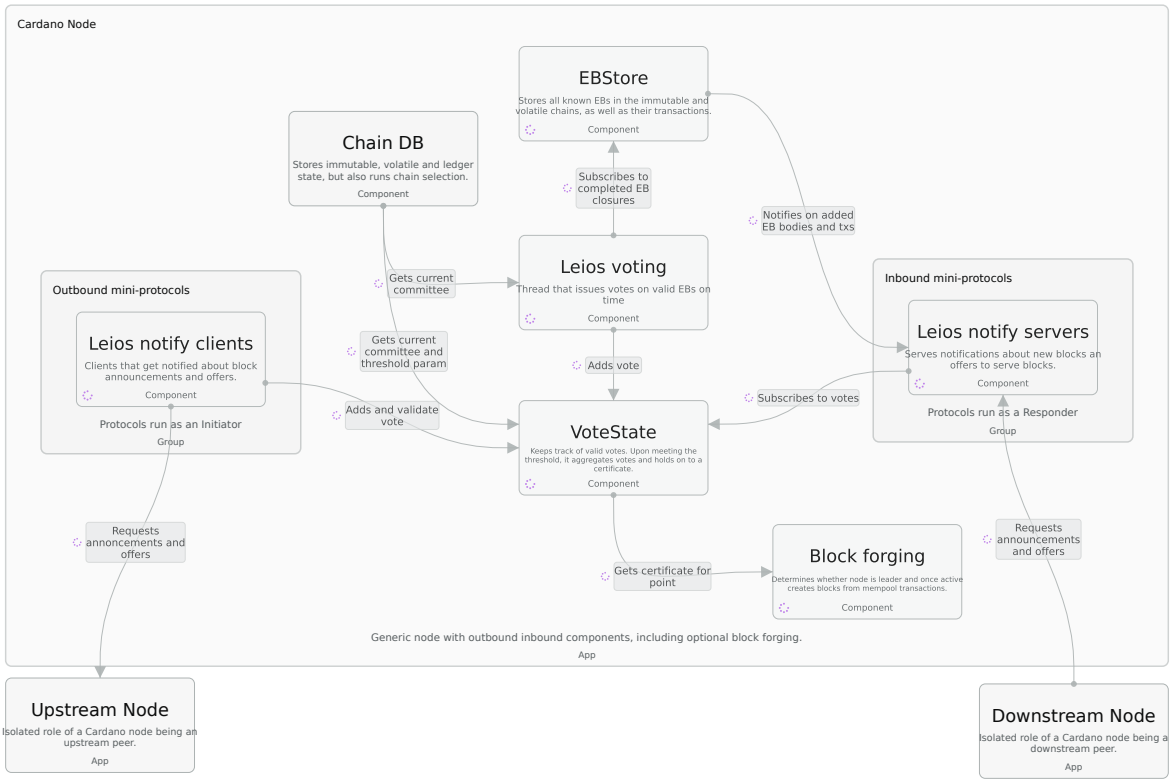
> [!WARNING]
>
> FIXME: The following paragraph (moved from the former Implementation notes section) overlaps with the prose above and should be deduped in the next pass.

The first version of LeiosTxCache should reliably cache all relevant transactions that are less than an hour or so oldthat age spans 180 active slots on average. A transaction is born when its oldest containing EB was announced or when it *entered* the Mempool (if it hasnt yet been observed in an EB). (Note that that means some txs age in the LeiosTxCache can increase when an older EB that contains it arrives.) Simple index maintained as a pair of priority queues (index and age) in manually managed fixed size bytearrays, backed by a double-buffered mmapped file for the transactions serializations. Those implementation choices prevent the sheer number of transactions from increasing GC pressure (adversarial load might lead to a ballpark number of 131000 transactions per hour), and persistences only benefit here would be to slightly increase parallelism/simplify synchronization, since persistence would let readers release the lock before finishing their search.

[!WARNING]

TBD: Is this motivating to *not* implement high churn components like the transaction cache in a garbage collected language and instead rely on implementations with more control over memory allocations? For example using an off-the-shelf key-value store for the transaction cache, or implementing a custom one in Rust and exposing it via FFI?

6.4.6 Voting and certification



[!WARNING]

FIXME: Discuss which committee to determine eligibility and validity of votes? refer to commitee selection in ledger chapter

A new thread dedicated to Leios vote production (**NEW-LeiosVoteProductionThread**) will wake up when the closure of an EB is newly available. If the voting rules would require the stake pool to vote (now or soon) for this EB if its valid, then this thread will begin validating it. Note if multiple closures arrive simultaneously, at most one of them could be eligible for a vote, since the voting rules require the EB to be announced by the tip of the nodes current selection. If the validation succeeds while the voting rules still require the stake pool to vote for this EB (TODO even if it has since switched its selection?), the thread will issue that vote.

- **REQ-IssueLeiosVotes** The node must vote for EBs exactly according to the rules from the CIP.
- **REQ-DiffuseLeiosVotes** The node must diffuse votes (via the Network layers mini-protocols) at least until theyre old enough that there remains only a negligible probability they could still enable an RB that was issued on-time to include a certificate for the EB they support.

A new storage component (**NEW-LeiosVoteStorage**) will store all votes received by a node, up to some conservative age (eg ten minutes). As votes arrive, they will be grouped according to the RB they support. When enough votes have arrived for some RB, the certificate can be generated immediately, which can avoid delaying the potential subsequent issuance of a CertRB by this node. A vote for the EB announced by an RB is irrelevant once all nodes will never switch their selection away from some block that is not older than that RB. This condition is very likely to be satisfied relatively soon on Cardano mainnet, unless its Praos growth is being disrupted. Therefore, the vote storage component can simply discard votes above some conservative age, which determines a stochastic upper bound the heap size of all sufficiently-young votes.

Once enough votes have been collected for an EB, a certificate can be formed and included in a ranking block:

- **REQ-IncludeLeiosCertificates** The node must include a certificate in each RB it issues if it has seen enough votes supporting the EB announced by the preceding RB. (TODO excluding empty or very nearly empty EBs?)

6.4.7 Chain selection

Leios does not change which chain the node prefers; it still must select the best chain it has acquired for the same definition of *best* as today. Leios does, on the other hand, change the definition of whether a chain is *acquired*, because a chain that includes CertRBs is not acquired before those certified EBs are acquired. The existing ChainSel logic already includes a similar corner-case, because RBs might arrive out of order (at least when syncing). Not being able to select a CertRB because its certified EB is still missing is directly comparable to not being able to select an RB because its predecessor RB is still missing. That same ChainSel mechanism will be enriched to also understand EB dependencies (**UPD-LeiosChainSelAndVolDB**).

- The VolDB in-memory index will also record for each RB in the VolDB whether that RB contains a Leios certificate.
- ChainSel/VolDB will be provided access to an in-memory index of which EB closures have been acquired.
- When ChainSel queries the VolDB for the set of candidate chains to consider for immediate selection, the VolDB will exclude CertRBs that certify an EB whose closure has not yet been acquired. That filter also requires an in-memory index of which EB closure is announced by each RB in the VolDB. (Note that that set of RBs might include some that were not (TODO yet?) processed as announcements by this nodes Leios logic, since the

Leios logic does bounded work per election. Hence it seems reasonable for the VolDB itself to own and maintain that index.)

- A new ChainSel trigger will allow the arrival of an EB closure to retrigger ChainSel for CertRBs that are unblocked by that arrival. (The similar trigger for when the Genesis implementations Limit on Eagerness (LoE) advances is a precedent.)

In Block Diffusion Pipelining via Deferred Validation, the node sometimes sends the MsgRollForward for an RBs header when ChainSel begins to fully process the RB. In the case of a CertRB, the downstream ChainSync peers will interpret that MsgRollForward as an offer of its predecessors EB closure. Note that that is safe only because ChainSel does not attempt to fully process the CertRB before it has the EB closure. Were that not true, then this MsgRollForward might cause an honest downstream peer to request a closure this node doesnt have, which would cause this node to disconnect from that honest peer.

6.4.8 Block validation

6.4.8.1 Prompt validation of CertRBs Because a CertRB is nothing more than the RB header paired with a certificate, that CertRB is valid if and only if its header and certificate are valid. The node only fetches an RB if it has already validated its header, and validating the header requires the forecasted LedgerView, so if the RB is present, then it must be within the nodes LedgerView forecast range. (TODO thats also true for an RB the node itself issued.) The forecasted LedgerView is also sufficient for the validation of the certificate, since the corresponding Leios committee is fixed by one of the Ledgers `mark` or `set` snapshots. Thus, when a CertRB arrives, the node can immediately validate it even if the node has not yet downloaded all of the predecessor RBs and certified EBs that would be necessary to construct the CertRBs ledger state. A certificate may also not cross an epoch boundary: ticking into a new epoch retires the pending announcement, so a CertRB is only ever judged against the committee of its own epoch and the first RB of an epoch never carries a certificate.

ChainSel cannot *completely process* the CertRB before those blocks arrive, because ChainSel must also produce the CertRBs resulting ledger state, which requires its preceding ledger state, which requires those blocks. But whether or not the certificate is valid can be immediately judged. Thats useful to do as soon as possible, because it may be the first and possibly even the only indication that allows the LeiosFetch logic to recover from having seen the wrong announcement first due to a violation of L_{hdr} or even having seen no timely announcement at all. Since this validated-certificate signal can be the only thing that triggers LeiosFetch to fetch the certified EB, if the signal could not arise until that EB were already fetched, the EB would never be fetched at all. Validating the certificate without the EB is precisely what breaks that circularity.

6.4.8.2 Proper application of CertRBs

[!WARNING]

FIXME: CertRB validity = process of cert verification, resolving the closure, applying all the transactions (UTxO-HD!) and then only validating the CertRB block itself

FIXME: Dont inline transactions into the block for the ledger, but instead propose a design that invokes ledger (via `ApplyTx`) separately in block validation. Alternative: new ledger rule for EBBODY validation (see note below). Moved here from the former Ledger / New block structure section.

The LedgerDB (**UPD-LeiosLedgerDb**) will need to retrieve the certified EBs closure from the LeiosEbStore when applying a CertRB. Due to **UPD-LeiosChainSelAndVolDB**, it should be impossible for that retrieval to fail.

In Praos, all transactions to be verified and applied to the ledger state are included directly in the block body. In Leios, ranking blocks (RB) may not include transactions directly, but instead certificate and reference to an endorsement block (EB) that further references the actual transactions. This gives rise to the following requirements:

- **REQ-LedgerResolvedBlockValidation** When validating a ranking block body, the ledger must be provided with all endorsed transactions resolved.
- **REQ-LedgerUntickedEBValidation** When validating a ranking block body, the ledger must validate endorsed transactions against the ledger state before updating it with the new ranking block.
- **REQ-LedgerTxValidationUnchanged** The actual transaction validation logic should remain unchanged, i.e. the ledger must validate each transaction as it does today.

The endorsement block itself does not contain any additional information besides a list of transaction identifiers (hashes of the full transaction bytes). Hence, the list of transactions is sufficient to reconstruct the EB body and verify the certificate contained in the RB. The actual transactions to be applied to the ledger state must be provided by the caller of the ledger interface, typically the storage layer.

![NOTE]

The EB actually contains transaction hashes + validity information. This is because block producers must be able to endorse / include transactions that fail phase-2 validation and consume collateral (thus be rewarded for this work without spending the input). This is consistent with what is stored in a Praos block today and the consensus/ledger interface should cover this already.

![WARNING]

TODO: update the EB CDDL in the CIP with this fact about the tx validity information

For the first version of the LedgerDB, it need not explicitly store EBs ledger state; the CertRBs result ledger state will reflect the EBs contents. A second version could thunk the EBs reapplication alongside the announcing RB, which would only avoid reapplication of one EB on a

chain switch (might be worth it for supporting tiebreakers?). The first version of LedgerDB can simply reapply the EBs transactions before tick-then-applying a CertRB. A second version should pass the EBs transactions to the ledger function (or instead the thunk of reapplying the EB)?

6.4.9 Catching up

> [!WARNING]
>
> TODO: Write about a more efficient way to catch up than to deal with out of order chain selection

6.5 Ledger

> [!WARNING]
>
> TODO: Mostly content directly taken from impact analysis. Expand on motivation and concreteness of changes.

The Ledger is responsible for validating Blocks and represents the actual semantics of Cardano transactions. CIP-164 sketches a protocol design that does not change the semantics of Cardano transactions, does not propose any changes to the transaction structure and also not requires changes to reward calculation. The ledger component has three main entry points:

1. Validating individual transactions via `LEDGER`
2. Validating entire block bodies via `BBODY`
3. Updating rewards and other ledger state (primarily across epochs) via `TICK`

The first will not need to change functionally, while the latter two will need to be updated to handle the new block structure (ranking blocks not including transactions directly) and to enable the determination of a voting committee for certificate verification. Any change to the ledger demands a hard-fork and a change in formats or functionality are collected into ledger eras. The changes proposed by CIP-164 will need to go into new ledger era:

- **REQ-LedgerNewEra** The ledger must be prepared with a new era that includes all changes required by CIP-164.

See Era and hard-fork coordination for a discussion on which era to target. For the remainder of this document, lets assume the changes will go into the `Dijkstra` era.

- **RSK-LeiosLedgerOverheadLatency**: Parsing a transaction, checking it for validity, and updating the ledger state accordingly all utilize CPU and heap (and also disk bandwidth with UTxO/Ledger HD). Frequent bursts of that resource consumption proportional to 15000% of a full Praos block might disrupt the caught-up node in heretofore unnoticed ways. Only syncing nodes have processed so many/much transactions in a short duration, and latency has never been a fundamental priority for a syncing node. Disruption of

the RTS is the main concern here, since there is plenty of CPU availablethe ledger is not internally parallelized, and only ChainSel and the Mempool could invoke it simultaneously.

6.5.1 Transaction validation levels

Validating individual transactions is currently done either via `applyTx` or `reapplyTx` functions. This corresponds to two levels of validation:

- `applyTx` fully validates a transaction, including existence of inputs, checking balances, cryptographic hashes, signatures, evaluation of plutus scripts, etc.
- `reapplyTx` only check whether a transaction applies to a ledger state. This does not include expensive checks like script evaluation (a.k.a phase-2 validation) or signature verification.

Where possible, `reapplyTx` is used when we know that the transaction has been fully validated before. For example when refreshing the mempool after adopting a new block. With Leios, a third level of validation is introduced:

- **REQ-LedgerTxNoValidation** The ledger should provide a way to update the ledger state by just applying a transaction without validation.

This third way of updating a ledger state would be used when we have a valid certificate about endorsed transactions in a ranking block. To avoid delaying diffusion of ranking blocks, we do want to do the minimal work necessary once an EB is certified and ease the protocol security argument with:

- **REQ-LedgerCheapReapply** Updating the ledger state without validation must be significantly cheaper than even reapplying a transaction is today.

Note that this already anticipates that the new, third level `notValidateTx` will be even cheaper than `reapplyTx`. Existing benchmarks indicate that `reapplyTx` is already at least one order of magnitude cheaper than `applyTx` for transactions.

6.5.2 New protocol parameters

CIP-164 introduces several new protocol parameters that may be updated via on-chain governance. The ledger component is responsible for storing, providing access and updating any protocol parameters. Unless some of the new parameters will be deemed constant (a.k.a globals to the ledger), the following requirements must be satisfied for all new parameters:

- **REQ-LedgerProtocolParameterAccess** The ledger must provide access to all new protocol parameters via existing interfaces.
- **REQ-LedgerProtocolParameterUpdate** The ledger must be able to update all new protocol parameters via on-chain governance.

Concretely, this means defining the `PParams` and `PParamsUpdate` types for the `Dijkstra` era to include the new parameters, as well as providing access via the `DijkstraPParams` and other type classes. The parameters, their `protocol_param_update` keys and the `PParams` lenses the ledger exposes them through are:

Key	Parameter	Symbol	Type
40	`ppLeiosAnnouncementPeriodLength`	L_{hdr}	`Milliseconds32`
41	`ppLeiosVotePeriodLength`	L_{vote}	`Milliseconds32`
42	`ppLeiosDiffusionPeriodLength`	L_{diff}	`Milliseconds32`
43	`ppLeiosCommitteeSize`	N_c	`uint .size 2`
44	`ppLeiosQuorumStakeThreshold`		`unit_interval`
45	`ppMaxEndorserBlockReferencesSize`	S_{EB}	`uint .size 4`
46	`ppMaxEndorserBlockTxsSize`	$S_{\text{EB-tx}}$	`uint .size 4`
47	`ppMaxEndorserBlockExUnits`	-	`ex_units`
48	`ppMaxRefScriptSizePerEndorserBlock`	$S_{\text{EB-ref}}$	`uint .size 4`

Note that the three timing parameters are reasoned about in seconds throughout CIP-164 that is the unit the protocols timing arguments use but are encoded in milliseconds so that sub-second values stay expressible independently of `slotLength`. L_{hdr} is the *announcement* period: an RB header is the announcement, so L_{hdr} is the time one announcement needs to travel the network and $3L_{\text{hdr}}$ is the window that rules out equivocation. The per-EB Plutus step and memory budgets share a single `ex_units` pair, mirroring the existing `maxTxExUnits` and `maxBlockExUnits`.

6.5.3 Key registration and rotation

Leios uses a per-pool BLS12-381 key to cast votes on endorser blocks and to build the aggregate certificate that anchors a certified EB in a ranking block. Because a node skips re-validating the endorsed transactions once it accepts a valid certificate (see Certificate verification), the integrity of these keys is chain-safety relevant: control of enough committee key material is enough to manufacture a certificate for transactions that were never validated.

- **REQ-RegisterBLSKeys** SPOs must be able to register a BLS key so their stake can be represented on the voting committee.

The registered key material is a public key together with a proof of possession, `bls_key = [bls_pubkey: G2 (96 bytes), bls_possessionproof: G1 (48 bytes)]` (using the small-signature BLS variant; the proof of possession is discussed under Committee selection). It is carried in an optional `bls_key` field of the existing **stake pool registration certificate**, rather than by a dedicated certificate or a block-header field.

Extending the pool registration certificate binds the voting keys lifecycle to the pools registered, re-registered and retired together and reuses the cold key authorization already required to

register a pool, so it needs no new certificate type, ledger rule or witnessing scheme. Because only stake pools sit on the committee, the key is intrinsically tied to a pools identity and stake, and belongs with the pools other registered key material. A dedicated certificate was weighed against it and would also work: it restates nothing, so an operator rotating quarterly cannot silently change margin, cost, pledge or relays by omission the one real cost of re-registering. That convenience did not outweigh a new certificate type with its own lifecycle to specify, test and maintain, so Dijkstra extends the pool certificate.

Carrying the key in the **block header**, signed by the KES key the way the VRF key is, was considered and rejected. It would let rotation ride the operational-certificate path and avoid a transaction entirely, but the key and its proof cannot sit in every header 144 bytes of dead weight on the most latency-critical object in the protocol so headers would need two operational-certificate shapes and a rule for switching between them. It is also unclear what an invalid proof of possession should mean there: rejecting the block punishes consensus for a registration mistake, accepting it lets a header assert a registration that did not happen. It would additionally require giving the ledger access to header data it does not process today, and the premise that operators would balk at a periodic transaction did not hold.

Rotation. A BLS key is a long-lived signing secret whose compromise is chain-safety relevant, so it must be rotated regularly. The right reference point is the KES *key rotation* cadence (~90 days), not KES *key evolution* (~12 hours): evolution provides forward secrecy within a single keys lifetime and is a separate mechanism that Leios does not require. A KES-like rotation cadence implies giving BLS keys a comparable time-to-live (~90 days), after which a freshly registered key must have taken over.

Unlike a KES key, which takes effect immediately on the chain it is used to build, a BLS key must be slot-activated at an epoch boundary, because the committee is fixed once per epoch from the stake distribution. In this respect BLS keys behave like VRF keys: rotatable at most once per epoch and only effective after a stake snapshot delay. A rotated key becomes active for voting when the snapshot in which it appears becomes the active stake distribution.

- **REQ-RotateBLSKeys** A registered BLS key must be rotatable by re-registration and activated at an epoch boundary

Expiry. Rotation is only a cadence if something enforces it, so a registered key is honoured for a bounded number of epochs after its registration and the pool falls back to a keyless seat once it lapses. The bound is *derived*, not governed: `maxKeyAgeEpochs = ceil(slotsPerKESPeriod * maxKESEvolutions / epochLength) + 2`, i.e. the KES key lifetime rounded up to whole epochs plus the two epochs of activation delay, which evaluates to **21 epochs** on mainnet. Deriving it from the KES setup already fixed in genesis keeps voting-key and operational-key rotation on one schedule by construction, with no second knob that could drift out of step; retuning the cadence means changing the KES setup, which requires a hard fork anyway. Expiry is judged against the epoch the committee is selected *for*, not the epoch its snapshot was taken in, so an epochs committee has a stable set of usable keys throughout. This is already implemented: `maxKeyAgeEpochs` in `Cardano.Ledger.Dijkstra.Rules.Snap` derives the bound from `Globals`

and seats the committee from the rotating mark snapshot, and `honouredBlsKey` in the formal ledger specification is its specification counterpart.

- **REQ-ExpireBLSKeys** A registered BLS key must stop being honoured `maxKeyAgeEpochs` epochs after its registration, taking effect at an epoch boundary, with the bound derived from the genesis KES parameters rather than governed.

Aligning BLS activation with VRF is a deliberate choice rather than a necessity, and BLS could in fact activate sooner. VRF keys are held back for two epochs precisely so that the leader-election nonce cannot be ground; BLS keys carry no such concern, so a registered BLS key could become active a full epoch earlier as soon as the mark stake snapshot it appears in has stabilised (after the `3k/f` stability window), taking effect the next epoch. We forgo that earlier activation and instead pin BLS to VRFs two-epoch schedule, because activation is not only an on-chain event: the operator must place the newly-active signing key on the hot, block-producing machine for the epoch in which it takes effect. If BLS activated an epoch before VRF, an SPO that rotated both keys in epoch `e` would face two separate hot-key swaps the BLS key in `e+1` and the VRF key in `e+2`. Aligning the two collapses this to a single swap in `e+2`; we accept the longer dead time the BLS key sits registered but inactive for an extra epoch in exchange for the more convenient, single-event key handling.

Either way, a freshly registered or rotated key is not eligible to vote until the snapshot in which it appears becomes the active stake distribution. A committee that must be live from the very first epoch (e.g. a fresh devnet) therefore needs the key seeded into the initial snapshots directly.

6.5.4 Committee selection

The voting committee for an epoch is chosen from the active stake distribution, weighted by stake, and fixed for the whole epoch.

- **REQ-StakeBasedCommitteeSelection** The committee must be selected deterministically from the stake distribution so that all nodes agree on both membership and seat assignment.

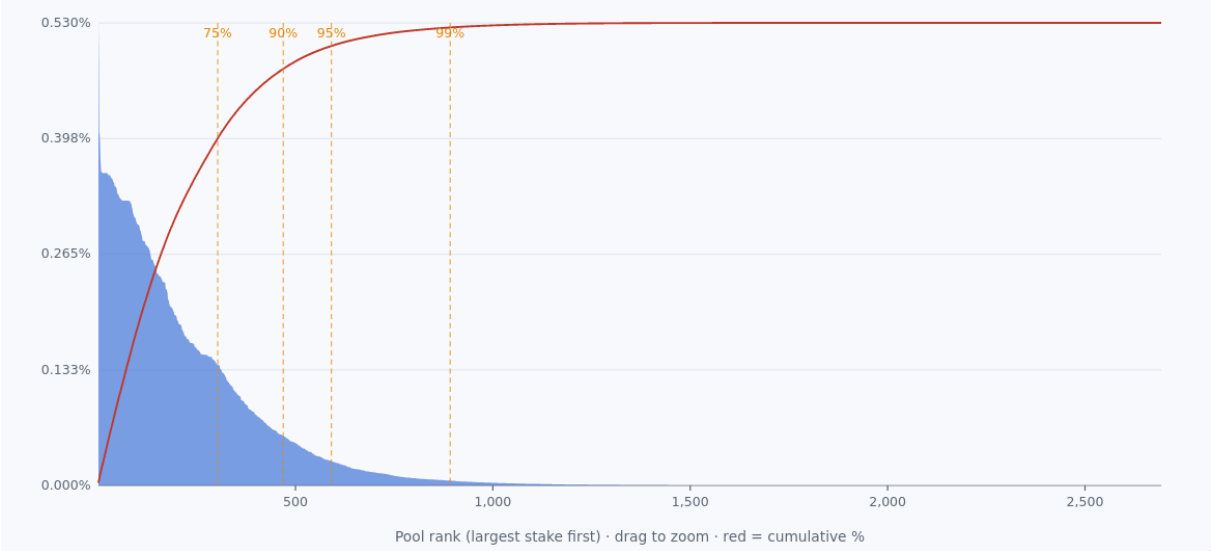
This is realized from the stake pool distribution the ledger already maintains (`PoolDistr`, stored in the ledger state as `nesPd`). Since the BLS key is threaded through `PoolDistr` every `IndividualPoolStake` carries an `individualPoolStakeBls :: StrictMaybe BlsKey` alongside its stake and VRF key that single structure is sufficient input for committee derivation, and no new snapshot machinery is needed. Derivation is a pure function of `PoolDistr`, so every node computes an identical committee.

The committee is **materialized once per epoch and stored in the ledger state**, rather than recomputed on each use. This is not merely an optimization: over an epoch a node may have to validate millions of votes against the committee, each needing to resolve a `voter_id`/seat to its stake weight and public key. Recomputing (ordering and truncating `PoolDistr`) per lookup would be prohibitive, so the committee is kept as a ready-to-index structure and refreshed at the

epoch boundary. Being part of the ledger state, it is also stored in ledger snapshots (and hence on disk under Ledger-HD) and exposed through the query interface (see Certificate verification).

Committee representation. A committee is an ordered sequence of *seats*, indexed `0..N-1`. Each seat records the pools relative stake (its voting weight) and its `StrictMaybe BlsKey`. The seat index is the `voter_id` a vote carries and the bit position addressed by a certificates signer bitfield, so the ordering must be canonical: pools are ordered by descending stake, ties broken by pool id. A seats index is thereby a deterministic function of `PoolDistr` alone.

Selection. Sorts the `PoolDistr` entries descending by stake, breaking ties by ascending pool id, and seats the first `leiosCommitteeSize` of them (or all pools, if fewer are registered). The parameter is a plain seat count N_c. CIP-164 governs the committee by size rather than by a cumulative-stake coverage target, and the ledger carries it as protocol parameter 43. On the current mainnet stake distribution the curve is strongly concentrated and long-tailed: of the ~2,700 registered pools, roughly 900 already cover 99% of active stake (and ~580 cover 95%), while the remaining long tail contributes the last percent. A feasible value for N_c is therefore around 900 seats, which is also what CIP-164 records as the feasible value.



[!NOTE]

The shape of this parameter was an open question cumulative-stake coverage vs. a fixed committee size vs. a hybrid cap. It is settled: a fixed committee size. See the effect of stake-distribution changes below for what that choice does and does not cost.

Effect of stake-distribution changes. Because votes are stake-weighted and each seats weight is recorded, certification always evaluates the *actual* signed stake the sum of the weights of the seats whose bits are set against the quorum threshold. There is therefore no under-representation in the sortition sense: the committee cannot misrepresent how much stake voted, whatever its size. A shift in the stake distribution changes only how much stake the committee *covers*. Under the chosen fixed committee size the vote count per EB is pinned and the covered

stake (N_c) floats with the distribution: a flatter distribution means the top N_c pools hold less of the total, a more concentrated one means they hold more. (A cumulative-stake coverage parameter would trade this the other way, pinning covered stake and letting the vote count float.) This makes N_c a **liveness** parameter, not a safety-critical one: if (N_c) drifts below the quorum threshold , no certificate can be formed even with unanimous support, but no certificate can ever be formed that misstates the stake behind it. Governance must therefore keep (N_c) > with headroom for members that are offline, keyless, or too slow to vote within L_{vote}.

Keyless seats. Committee membership is by stake, independent of key registration: a stake-selected pool whose `individualPoolStakeBls` is `SNothing` still occupies a seat, but that seat is *keyless* it holds a weight yet has no key to sign with. A keyless seat can never contribute to a valid certificate, and a certificate whose signer bitfield sets a keyless seats bit must be rejected. Because keyless seats fall out of the same deterministic derivation from `PoolDistr`, every node agrees on exactly which seats are keyless; a divergence would make honest certificates appear invalid (the `SignerWithoutKey` failure).

- **REQ-KeylessSeat** The committee must be able to represent keyless seats, such that committee selection is independent of key registration.

Proof-of-possession checks. BLS aggregate signatures are vulnerable to rogue-key attacks, in which an adversary registers a key crafted relative to others keys so that an aggregate appears to include victims who never signed. A `BlsKey` therefore carries a proof of possession over its public key, verified when the key is registered so that a key already resident in `PoolDistr` can be trusted without re-checking it on every certificate. A registration whose proof does not verify is rejected and never reaches `PoolDistr`; equivalently, a seat without a valid proof is treated as keyless.

- **REQ-CheckProofOfPossession** A committee seat with an invalid proof of possession must be treated as keyless.

6.5.5 Certificate verification

A CertRB that anchors a certified EB carries a Leios certificate. Concretely the Dijkstra block body gains an optional certificate (`Maybe LeiosCert`) which is set only on such a block. The certificate is compact: a **bitfield** identifying which committee seats signed, and a single **aggregate BLS signature** standing in for their individual votes for the endorser block (its exact contents and encoding are covered under Serialization).

Verifying a certificate requires the committee for the relevant epoch. That committee is derived deterministically from the stake distribution already held in the ledger state (see Committee selection), so no new inputs are needed and because keys are registered through transactions (see Key registration and rotation), the ledger already holds every public key in its state and does **not** need access to block headers. Being part of the ledger state, the committee is also stored in ledger snapshots (and hence on disk under Ledger-HD) and can be exposed through the

existing query interfaces, which components also use to check individual votes before diffusing them.

No certification across boundaries. What makes the relevant epoch above a single epoch rather than a choice is that a certificate may not cross an epoch boundary (CIP-164 Step 5). That is a simplification rather than a safety requirement: the epoch boundary already concentrates a great deal of work, and an announcement surviving it would be one more thing to carry across while certifying across an *era* boundary would further oblige the announcement, the certificate and the EB to stay syntactically compatible with the very format change a new era exists to make, at real cost every time those formats do change. Era transitions fall on epoch boundaries, so the epoch rule covers both, and it costs at most one EB per boundary.

- **REQ-LedgerStateVotingCommittee** The Leios voting committee must be part of the ledger state, materialized and updated on epoch boundaries, and queryable through existing interfaces.

Verification then reconstructs the signing public keys from the bitfield rejecting the certificate if any signalled seat is keyless aggregates those keys, checks the aggregate signature, and confirms the signing seats clear the required stake threshold.

- **REQ-LedgerCertificateVerification** The ledger must verify a certificate contained in a ranking block against the voting committee derived from its state, treating a certificate that references a keyless seat, fails signature verification, or falls short of the stake threshold as invalid.

Certificate verification and block-body application are coupled. A valid certificate is precisely what authorizes a node to incorporate an EBs transaction closure *without* re-validating it (see Transaction validation levels): the closure must be applied for the resulting ledger state to be correct, while the certificate is what makes applying it without validation safe. The `BBODY` rule is therefore extended so that, when a ranking block carries a certificate, it verifies the certificate against the committee and, on success, applies the endorsed closure through the cheap non-validating path.

> [!NOTE] This requires the EB closure to be available to `BBODY`, and leaves open how much of the work lives in `BBODY` itself versus a dedicated EB-body rule, and the precise ordering of applying the closure relative to verifying the certificate. That depends on the consensus/ledger interface for block validation and is still being settled.

6.5.6 Serialization

Traditionally, the ledger component defines the serialization format of blocks and transactions. CIP-164 introduces three new types that need to be serialized and deserialized:

[!WARNING]

TODO: Serialization of votes a consensus component responsibility?

FIXME: out of these, only the RB / block body is currently in the realm of ledger.

- **REQ-LedgerSerializationRB** The ranking block body contents must be deterministically de-/serializable from/to bytes using CBOR encoding.
- **REQ-LedgerSerializationEB** The endorsement block structure must be deterministically de-/serializable from/to bytes using CBOR encoding.
- **REQ-LedgerSerializationVote** The vote structure must be deterministically de-/serializable from/to bytes using CBOR encoding.

Corresponding types with instances of `EncCBOR` and `DecCBOR` must be provided in the ledger component. The `cardano-ledger` package is a dependency to most of the Haskell codebase, hence these new types can be used in most other components.

6.5.7 Cryptography

[!WARNING]

FIXME: The last three sections in ledger about key registration / rotation, committee selection and certificate verification are very crypto heavy and arguing for one or the other scheme in this document is wrong. It would need to be in the CIP-164 anyways. Thus, revamp the whole section and/or merge it with the above sections. Keep only interesting / relevant bits like: a new package `cardano-crypto-leios` is added to `cardano-base` to implement the cryptographic operations on certificates (aggregate + verify) following the CIP.

TODO: Mostly content directly taken from impact analysis. Expand on motivation and concreteness of changes.

The security of the votes cast and the certificates that Leios uses to accept EB blocks depends on the usage of the pairing-based BLS12-381 signature scheme (BLS). This scheme is useful, as it allows for aggregation of public keys and signatures, allowing a big group to signal their approval with one compact artifact. Besides Leios, it is also likely that Peras will use this scheme.

This section derives requirements for adding BLS signatures to `cardano-base` and sketches changes to satisfy them. The scope is limited to cryptographic primitives and their integration into existing classes; vote construction/logic is out of scope. This work should align with this IETF draft.

Note that with the implementation of CIP-0381 `cardano-base` already contains basic utility functions needed to create these bindings; the work below is thus expanding on that. The impact of the below requirements thus only extends to this module and probably this outward facing class.

6.5.7.1 Core functionality

The following functional requirements define the core BLS signature functionality needed:

- **REQ-BlsTypes** Introduce opaque types for `SecretKey`, `PublicKey`, `Signature`, and `AggSignature` (if needed by consensus).
- **REQ-BlsKeyGenSecure** Provide secure key generation with strong randomness requirements, resistance to side-channel leakage.
- **REQ-BlsVariantAbstraction** Support both BLS variantssmall public key and small signaturebehind a single abstraction. Public APIs are variant-agnostic.
- **REQ-BlsPoP** Proof-of-Possession creation and verification to mitigate rogue-key attacks.
- **REQ-BlsSkToPk** Deterministic sk pk derivation for the chosen variant.
- **REQ-BlsSignVerify** Signature generation and verification APIs, variant-agnostic and domain-separated (DST supplied by caller). Besides the DST, the interface should also implement a per message augmentation.
- **REQ-BlsAggregateSignatures** Aggregate a list of public keys and signatures into one.
- **REQ-BlsBatchVerify** Batch verification API for efficient verification of many `(pk, msg, sig)` messages.
- **REQ-BlsDSIGNIntegration** Provide a `DSIGN` instance so consensus can use BLS via the existing `DSIGN` class, including aggregation-capable helpers where appropriate.
- **REQ-BlsSerialisation** Deterministic serialisation: `ToCBOR`/`FromCBOR` and raw-bytes for keys/signatures; strict length/subgroup/infinity checks; canonical compressed encodings as per the Zcash standard for BLS points.
- **REQ-BlsConformanceVectors** Add conformance tests using test vectors from the initial Rust implementation to ensure cross-impl compatibility.

6.5.7.2 Performance and quality

[!WARNING]

TODO: Move to performance or testing sections?

The following non-functional requirements ensure the implementation meets performance and quality standards:

- **REQ-BlsPerfBenchmarks** Benchmark single-verify, aggregate-verify, and batch-verify; report the impact of batching vs individual verification.
- **REQ-BlsRustParity** Compare performance against the Rust implementation; document gaps and ensure functional parity on vectors.
- **REQ-BlsDeterminismPortability** Deterministic results across platforms/architectures; outputs independent of CPU feature detection.
- **REQ-BlsDocumentation** Document the outward facing API in cardano-base and provide example usages. Additionally add a section on dos and donts with regards to security of this scheme outside the context of Leios.

6.5.7.3 Implementation notes

Note that the PoP checks probably are done at the certificate level, and that the above-described API should not be responsible for this. The current code on BLS12-381 already abstracts over both curves `G1`/`G2`, we should maintain this. The `BLST` package also exposes fast verification over many messages and signatures + public keys by doing a combined pairing check. This might be helpful, though its currently unclear if we can use this speedup. It might be the case, since we have linear Leios, that this is never needed.

6.6 Node API

[!WARNING]

FIXME: Groups changes to the nodes external interfaces (N2C, queries, configuration) that dont fit neatly into the protocol-flow spine.

6.6.1 LocalStateQuery additions

[!WARNING]

FIXME: Write this. New queries for registered BLS keys, the voting committee, and any other Leios-related ledger state exposed to clients.

6.6.2 Node-to-client

[!WARNING]

TODO: concrete discussion on how the `cardano-node` will need to change on the N2C interface, based on client interfaces

- Mithril, for example, does use N2C `LocalChainSync`, but does not check hash consistency and thus would be compatible with our plans.

6.6.3 Feature flags and configuration

[!WARNING]

FIXME: Write this. Feature flags for dev phases and new configuration data exposed to operators.

6.7 End-to-end testing

> [!WARNING]
>
> FIXME: Moved out of Technical design pending a decision on its final home (candidates: under Performance and quality assurance strategy, or its own chapter).
>
> TODO: Mostly content directly taken from impact analysis. Expand on motivation and concreteness of changes. This also feels a bit ouf of touch with the implementation plan; to be integrated better for a more holistic quality and performance strategy.

The cardano-node-tests project offers test suites for end-to-end functional testing and mainnet synchronization testing.

The end-to-end functional test suite checks existing functionalities. It operates on both locally deployed testnets and persistent testnets such as Preview and Preprod. With over 500 test cases, it covers a wide spectrum of features, including basic transactions, reward calculation, governance actions, Plutus scripts and chain rollback.

Linear Leios primarily impacts the consensus component of `cardano-node`, leaving end-user experience and existing functionalities unchanged. Consequently, the current test suite can largely be used to verify `cardano-node`s operation after the Leios upgrade, requiring only minor adjustments.

6.7.1 New end-to-end tests for Leios

New end-to-end tests for Leios will focus on two areas:

- Hard-fork testing from the latest mainnet era to Leios
- Upgrading from the latest mainnet `cardano-node` release to a Leios-enabled release

6.7.2 New automated upgrade testing test suite

The suite will perform the following actions:

1. **Initialize Testnet** - Spin up a local testnet, starting in the Byron era, using the latest mainnet `cardano-node` release.
2. **Initial Functional Tests** - Run a subset of the functional tests.
3. **Partial Node Upgrade** - Upgrade several block-producing nodes to a Leios-enabled release.
4. **Mid-Upgrade Functional Tests** - Run another subset of the functional tests.
5. **Cooperation Check** - Verify seamless cooperation between nodes running the latest mainnet `cardano-node` release and those running a Leios-enabled release on the same testnet.
6. **Full Node Upgrade** - Upgrade the remaining nodes to a Leios-enabled release.

7. **Leios Hard-Fork** - Perform the hard-fork to Leios.
8. **Post-Hard-Fork Functional Tests** - Run a final subset of the functional tests.

6.8 Performance and tracing

[!WARNING]

FIXME: Integrate this with the implementation plan

6.8.1 Observability as a first-class citizen

By implementing evidence of code execution, a well-founded tracing system is the prime provider of observability for a system. This observability not only forms the base for monitoring and logging, but also performance and conformance testing.

For principled Leios quality assurance, a formal specification of existing and additional Leios trace semantics is being created, and will need to be maintained. This specification is language- / implementation-independent, and should not be automatically generated by the Nodes Haskell reference implementation. It needs to be an independent source of truth for system observables.

Proper emission of those traces true to their semantics will need to be ensured for all prototypes and diverse implementations of Leios.

Last not least, the existing simulations will be maintained and kept operational. Insights based on concrete evidence from testing implementation(s) can be used to further refine their model as Leios is developing.

6.8.2 Testing during development

Wheras simulations operate on models and are able to falsify hypotheses or assess probability of certain outcomes, evolving prototypes and implementations rely on evidence to that end. A dedicated environment suitable for both performance and conformance testing will be created; primarily as feedback for development, but also to provide transparency into the ongoing process.

This environment serves as a host for operating small testnets. It automates deployment and configuration, and is parametrizable as far as topology, and deployed binaries are concerned. This means it needs to abstract wrt. of configuring a specific prototype or implementation. This enables deploying adversarial nodes for the purpose of network conformance testing, as well as performance testing at system integration level. This environment also guarantees the observed network behaviour or performance metrics have high confidence, and are reproducible.

Conformance testing can be done on multiple layers. For authoritative end-to-end verification of protocol states, all evidence will need to be processed wrt. the formal specification, keeping

track of all states and transitions. A second, complementary approach we chose is conformance testing using Linear Temporal Logic (LTL). By formulating LTL propositions that need to hold for observed evidence, one can achieve broad conformance and regression testing without embedding it in protocol semantics; this tends to be versatile and fast to evaluate incrementally. This means, system invariants can be tested as part of CI, or even by consuming live output of a running testnet.

Performance testing requires constant submission pressure over an extended period of time. With Leios being built for high throughput, creating submissions fully dynamically (as is the case with Praos benchmarks) is likely insufficient to maintain that pressure. We will create a declarative, abstract definition of what constitutes a workload to be submitted. This enables to pre-generate part or all submissions for the benchmarks. Moreover, it guarantees identical outcomes regardless of how exactly the workload is generated. These workloads will retain their property of being customizable regarding particular aspects they stress in the system, such as the UTxO set, or Plutus script evaluation.

As raw data from a benchmark / conformance test can be huge, existing analysis tooling will be extended or built, such that extracting key insights from raw data can be automated as much as possible.

This requires the presence of basic observability with shared semantics of traces in all participating prototypes or implementations, as outlined in the previous section.

6.8.2.1 Micro-benchmarks Additionally, smaller units of implementation (vs. full system integration) also deserve a performance safeguard. We will create and executing benchmarks that target isolated components of the system, and do not need a full testnet to run. The aim of those microbenchmarks is to provide long-term performance comparability for those components. This entails that the benchmark input needs to be stable across versions; it also requires a stable hardware specification to execute those benchmarks on (dynamically allocated environments are not suitable for that purpose). Thus, these microbenchmarks will be automated on fixed hardware - which can be considered a calibrated measurement device - and their artifacts archived and conveniently exposed for feedback and transparency.

6.8.3 Testing a full implementation

This eventual step will stop support for prototypes and instead focus on full implementations of Leios. This will allow for a uniform way to operate, and artificially constrain, Leios by configuration while maintaining its performance properties.

Furthermore, this phase will see custom benchmarks that can scale individual aspects of Leios independently (by config or protocol parameter), so that the observed change in performance metrics can be clearly correlated to a specific protocol change. This also paves the way for testing hypotheses about the effect of protocol settings or changes based on evidence rather than a model.

7 Glossary

Term	Definition
RB	Ranking Block - Extended Praos block that announces and certifies EBs
EB	Endorser Block - Additional block containing transaction references
CertRB	Ranking Block containing a certificate
TxRB	Ranking Block containing transactions
BLS	Boneh-Lynn-Shacham signature scheme using elliptic curve cryptography
BLS12-381	Specific elliptic curve used in cryptography
PoP	Proof-of-Possession - Prevents rogue key attacks in BLS aggregation
$L_{\mathbf{hdr}}$	Header diffusion period (1 slot)
$L_{\mathbf{vote}}$	Voting period (4 slots)
$L_{\mathbf{diff}}$	Certificate diffusion period (7 slots)
FFD	Freshest-First Delivery - Network priority mechanism
ATK-LeiosProtocolBurst	Attack where adversary withholds and releases EBs simultaneously

8 References

[!WARNING] TODO: Use pandoc-compatible citations https://pandoc.org/MANUAL.html#citation-syntax

1. **CIP-164**: Ouroboros Linear Leios - Greater transaction throughput https://github.com/cardano-foundation/CIPs/tree/master/CIP-0164
2. **Leios Impact Analysis**: High-level component design https://github.com/input-output-hk/ouroboros-leios/blob/main/docs/ImpactAnalysis.md
3. **Leios Formal Specification**: Agda implementation https://github.com/input-output-hk/ouroboros-leios/tree/main/formal-spec
4. **Cardano Ledger Formal Specification**: https://github.com/IntersectMBO/formal-ledger-specifications/
5. **Ouroboros Peras Technical Design**: Finality improving consensus upgrade https://tweag.github.io/cardano-peras/peras-design.pdf
6. **Ouroboros Genesis**: Bootstrap mechanism https://iohk.io/en/research/library/papers/ouroboros-genesis-composable-proof-of-stake-blockchains-with-dynamic-availability/
7. **Ouroboros Network Specification**: https://ouroboros-network.cardano.intersectmbo.org/pdfs/networ spec/network-spec.pdf

8. **UTxO-HD Design**: https://github.com/IntersectMBO/ouroboros-consensus/blob/main/docs/website/developers/utxo-hd/index.md